

Multiagentní systémy

Učební text ke státní závěrečné zkoušce

SZZ Umělá inteligence, MFF UK

srpen 2026

Okruh „Multiagentní systémy“ magisterské SZZ Umělá inteligence. Kapitoly = věty oficiálního znění okruhu **jedna ku jedné** (mapovací tabulka níže). Zdroje: slidy NAIL106 *Multiagentní systémy* (Neruda, Pilát, MFF UK); M. Wooldridge: *An Introduction to MultiAgent Systems* (2. vyd., 2009); Y. Shoham, K. Leyton-Brown: *Multiagent Systems* (2009); S. Russell, P. Norvig: *Artificial Intelligence: A Modern Approach*. Anglické termíny kurzívou v závorce při prvním výskytu (zkoušející je používá, literatura je anglická).

Rozsah textu a značení

- Rozsah kapitol ~ váha látky na přednášce: nejvíc architektury agentů, komunikace, kooperace a teorie her.
- [♦ **nad rámec slidů**] = látka **mimo slidy** přednášky, ponechaná, protože ji *jmenuje okruh* nebo bez ní výklad nedává smysl; nabídka ke škrtnutí. Neoznačené = doloženo ve slidech.
- Kapitoly **3** (hledání cesty) a **6** (učení agentů): ve slidech nejsou (přednáška je uvádí mezi nepokrytými), okruh je vyjmenovává ⇒ nutné umět; zpracovány stručně.

Mapování okruhu na kapitoly

Věta oficiálního znění okruhu	Kapitola v tomto textu
Architektura autonomního agenta; mechanismus výběru akcí.	1 Architektura autonomního agenta a mechanismus výběru akcí
Metody pro řízení agentů; symbolické a konekcionistické reaktivní plánování, hybridní přístupy.	2 Metody pro řízení agentů
Problém hledání cesty.	3 Problém hledání cesty
Komunikace a znalosti v multiagentních systémech, ontologie, řečové akty, FIPA-ACL, protokoly.	4 Komunikace a znalosti v MAS
Distribuované řešení problémů, kooperace, Nashova rovnováha, Paretova efektivita, alokace zdrojů, aukce.	5 Distribuované řešení problémů, kooperace, teorie her a aukce
Metody pro učení agentů; zpětnovazební učení.	6 Učení agentů a zpětnovazební učení
Metodologie návrhu, jazyky a prostředí multiagentních systémů.	7 Metodologie návrhu, jazyky a prostředí MAS

Obsah

Mapování okruhu na kapitoly	1
1 Architektura autonomního agenta a mechanismus výběru akcí	4
1.1 Agent a multiagentní systém	4
1.2 Vlastnosti inteligentního agenta	4
1.3 Intencionální systémy a intencionální postoj	5
1.4 Prostředí a jeho vlastnosti	5
1.5 Abstraktní architektura agenta	5
1.6 Čistě reaktivní agent a agent s vnitřním stavem	6
1.7 Utilita: jak agentovi říct, co má dělat	7
1.8 Predikátová specifikace úlohy	8

1.9	Mechanismus výběru akcí — přehled	8
1.10	Shrnutí ke zkoušce	9
2	Metody pro řízení agentů: symbolické a konekcionistické reaktivní plánování, hybridní přístupy	9
2.1	Klasický přístup UI a deduktivní agent	9
2.2	Praktické uvažování: architektura BDI	11
2.3	Plánování: STRIPS	11
2.4	Implementace BDI agenta	12
2.5	Odhodlání agenta (<i>commitment</i>)	13
2.6	Procedurální odvozovací systém (PRS)	13
2.7	Reaktivní přístup a jeho východiska	15
2.8	Symbolické reaktivní plánování: subsumpční architektura	15
2.9	Konekcionistické reaktivní plánování: agent network architecture	16
2.10	Omezení reaktivních architektur	17
2.11	Hybridní architektury	17
2.12	Komplexní architektury: IDA a globální pracovní prostor	18
2.13	Srovnání architektur	19
2.14	Shrnutí ke zkoušce	19
3	Problém hledání cesty	20
3.1	Formulace a zařazení	20
3.2	A*	21
3.3	Dynamické prostředí: D* Lite	21
3.4	Hledání cest pro více agentů (MAPF)	22
3.5	Shrnutí ke zkoušce	22
4	Komunikace a znalosti v MAS: ontologie, řečové akty, FIPA-ACL, protokoly	23
4.1	Od agenta k multiagentnímu systému	23
4.2	Co je ontologie	23
4.3	Ontologické jazyky	24
4.4	Uvažování o znalostech: epistemická logika	25
4.5	Proč je komunikace agentů jiná	25
4.6	Teorie řečových aktů	25
4.7	KQML a FIPA-ACL	26
4.8	Interakční protokoly	27
4.9	Shrnutí ke zkoušce	28
5	Distribuované řešení problémů, kooperace, teorie her a aukce	28
5.1	Koherence a koordinace	29
5.2	Sdílení úloh a sdílení výsledků	29
5.3	Contract Net	29
5.4	Systémy s tabulí (blackboard)	30
5.5	Distribuované omezující podmínky (DisCSP a DCOP)	31
5.6	Interakce agentů v prostředí	31
5.7	Sobecký agent	31
5.8	Hra v normální formě	31
5.9	Klasické hry	33
5.10	Sociální volba a volby	34
5.11	Aukce jako mechanismus alokace zdrojů	35
5.12	Čtyři základní typy aukcí	35
5.13	Kombinatorické aukce	36
5.14	Další typy aukcí a alokace zdrojů	36

5.15	Návrh mechanismů, VCG a vyjednávání	37
5.16	Shrnutí ke zkoušce	37
6	Metody pro učení agentů a zpětnovazební učení	38
6.1	Proč a jak se agenti učí	38
6.2	Markovský rozhodovací proces	39
6.3	Q-learning a SARSA	40
6.4	Metody založené na politice	41
6.5	Multiagentní zpětnovazební učení	41
6.6	Shrnutí ke zkoušce	41
7	Metodologie návrhu, jazyky a prostředí MAS	42
7.1	Agentově orientované softwarové inženýrství	42
7.2	Standard FIPA a architektura platformy	42
7.3	Jazyky a prostředí	43
7.4	Shrnutí ke zkoušce	44
	Závěrem: jak na zkoušku	44

1 Architektura autonomního agenta a mechanismus výběru akcí

1.1 Agent a multiagentní systém

Multiagentní systémy (*multiagent systems*, MAS) — samostatná oblast informatiky od 90. let. Hnací trendy:

- ubikvitnost výpočetní techniky; konektivita a distribuovanost (výpočet jako interakce);
- rostoucí složitost úloh; delegace úloh na software; tlak na snadnost použití;
- paradigma „vstup → výpočet → výstup → konec“ nestačí pro software trvale zasazený do prostředí, které se mění nezávisle na něm.

Agent — pracovní definice

Russell, Norvig (AIMA): cokoli, co *vnímá* prostředí senzory a *působí* na ně efekty.

Pattie Maes: výpočetní systém obývající složité dynamické prostředí, v němž autonomně vnímá a jedná, a tím plní cíle či úkoly, pro které byl navržen.

Wooldridge, Jennings: počítačový systém zasazený do prostředí, schopný v něm *autonomního jednání* vedoucího ke splnění delegovaných cílů.

Jednotná definice neexistuje — Franklin a Graesser (1996, *Is it an agent, or just a program?*): desítky nekompatibilních definic. Podstatné: agent $\neq$ procedura či objekt.

Multiagentní systém

Systém více agentů, kteří spolu *interagují* — nejčastěji výměnou zpráv po síti. Agenti *nemusí sdílet společný cíl*, mohou mít vlastní i protichůdné zájmy $\Rightarrow$ nutnost studovat vyjednávání, koordinaci a dynamické rozdělování zdrojů.

Autonomie. Člověk plně, metoda v Javě vůbec, agent mezi: volí *způsob* řešení (podcíle), ne hlavní cíl — ten je delegován.

Vymezení vůči příbuzným oborům (pět pohledů na MAS):

- **Softwarové inženýrství** — další stupeň abstrakce: strojový kód $\rightarrow$ objekty $\rightarrow$ distribuované objekty $\rightarrow$ agenti; bonmot „objekty to dělají zadarmo, agenti proto, že chtějí“.
- **Distribuované a paralelní systémy** — mutex, deadlock, konsensus vyřešeny; agenti jsou však *autonomní*, koordinace se řeší až za běhu.
- **Umělá inteligence** — tradičně $MAS \subseteq UI$; Russell a Norvig naopak: cíl UI = návrh inteligentních agentů. MAS užívá techniky UI + *sociální* aspekty, které klasická UI přehlíží.
- **Teorie her a ekonomie** — formální aparát racionálního rozhodování více subjektů; matematická teorie her řeší *existenci*, MAS *výpočetní* aspekty.
- **Sociologie** — společnosti agentů: inspirace ano, nutnost ne. Opačná vazba pevnější: sociologie, ekonomie i ekologie staví simulační modely na agentech (*agent-based modelling*, ABM).

1.2 Vlastnosti inteligentního agenta

- **Reaktivita** (*reactive*) — vnímá prostředí, reaguje na změny v rozumném (reálném) čase.
- **Proaktivita** (*proactive*) — vlastní cíle, naplňuje je z vlastní iniciativy, ne jen jako odpověď na podnět.
- **Sociálnost** (*social ability*) — komunikuje s jinými agenty i s lidmi.

Čistě reaktivní i čistě cílové chování = snadné; obtížná je **rovnováha reaktivity a proaktivity**: ani slepě držet plán, který ztratil smysl, ani stále přehodnocovat cíle a nikam nedojít. Vrací se: odhodlání agenta (odst. 2.5), hybridní architektury (odst. 2.11).

1.3 Intencionální systémy a intencionální postoj

Agentům přisuzujeme *mentální stavy* („agent věří, že...“, „agent chce...“). **Intencionální systém** (Dennett) — entita, jejíž chování lze predikovat přisouzením přesvědčení (*beliefs*), tužeb (*desires*) a racionality. Hierarchie: 1. řád = přesvědčení a touhy; 2. řád = přesvědčení a touhy o přesvědčeních a touhách (svých i cizích); atd.

Tři Dennettovy *postoje* (*stances*) k predikci chování:

- *fyzikální* — stačí fyzikální zákony (hozený kámen: hmotnost, rychlost, gravitace, ne touhy);
- *návrhový* (*design stance*) — neznám fyziku, znám účel (nastavím budík bez znalosti elektroniky);
- *intencionální* — popis pomocí přesvědčení, tužeb a racionality.

Shoham: „Vypínač je velmi kooperativní agent, který vede proud, kdykoli věří, že si to přejeme.“ Konzistentní, ale absurdní — *existuje jednodušší, stejně přesný popis*. Intencionální postoj se vyplatí, kde fyzikální či návrhový popis neexistuje nebo je nepraktický. **Pro informatiku: abstrakce pro zvládnutí složitosti** — pro mnoho programátorů hlavní přínos MAS.

1.4 Prostředí a jeho vlastnosti

Typ prostředí určuje chování agenta. Klasifikace (Russell, Norvig):

Dichotomie	Význam
plně / částečně pozorovatelné (<i>fully / partially observable</i>)	senzory dávají úplný stav prostředí — nebo jen část (skryté informace, šum, omezený dosah)
statické / dynamické (<i>static / dynamic</i>)	prostředí se mění jen akcemi agenta — nebo i jinak (jiní agenti, přírodní děje); ve statickém si agent může „v klidu rozmyslet“ tah
deterministické / nedeterministické	akce má právě jeden výsledek — nebo více (se známým či neznámým rozdělením pravděpodobnosti = stochastické prostředí)
diskrétní / spojitě	konečný počet stavů a akcí — nebo spojitě veličiny (poloha, rychlost)

Nepříjemné konstatování: zajímavá prostředí (reálný svět, internet) = spojitá, nedeterministická, dynamická, částečně pozorovatelná $\Rightarrow$ pevná tabulka pravidel nestačí.

Příklady. **Termostat:** vjemy „je zima“ / „teplota OK“, akce zapnout / vypnout; přesto nedeterministické prostředí (někdo otevře dveře). **Softwarový démon** (unixový *xbiff*): prostředí = operační systém, vjemy = systémová volání, akce = softwarové operace.

1.5 Abstraktní architektura agenta

Předpoklad: diskretní prostředí — neomezuje, spojitě lze diskretizovat s libovolnou přesností.

Prostředí, agent, běh

Prostředí: konečně mnoho diskretních stavů $E = \{e, e', \dots\}$. **Agent:** konečná množina akcí $A = \{a, a', \dots\}$.

Běh (*run*) — posloupnost střídajících se stavů prostředí a akcí:

$$r = e_0 \xrightarrow{a_0} e_1 \xrightarrow{a_1} e_2 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} e_n.$$

R — všechny konečné běhy (nad E a A); $R^A \subseteq R$ běhy končící akcí; $R^E \subseteq R$ běhy končící stavem prostředí.

Přechodová funkce prostředí (*state transformer function*)

$$\tau : R^A \rightarrow 2^E$$

Běh končící akcí $\mapsto$ množina možných následujících stavů. Důsledky:

- **prostředí má historii** — další stav závisí na celém běhu, ne jen na poslední akci;
- **prostředí je nedeterministické** — výsledkem je množina stavů, který nastane, dopředu nevíme.

$\tau(r) = \emptyset \Rightarrow$ běh skončil. Předpoklad: všechny běhy skončí.

Prostředí $Env = \langle E, e_0, \tau \rangle$, e_0 počáteční stav.

Agent a systém

Agent — funkce z běhů končících stavem prostředí do akcí:

$$Ag : R^E \rightarrow A.$$

Rozhoduje podle *celé historie* a *deterministicky*. $\mathcal{AG}$ — množina všech agentů.

Systém — dvojice $\langle Ag, Env \rangle$. Posloupnost $(e_0, a_0, e_1, a_1, \dots)$ je *běh agenta* Ag v prostředí Env , právě když

1. e_0 je počáteční stav Env ,
 2. $a_0 = Ag(e_0)$,
 3. pro každé $u > 0$: $e_u \in \tau((e_0, a_0, \dots, a_{u-1}))$ a $a_u = Ag((e_0, a_0, \dots, e_u))$.
- $R(Ag, Env)$ — množina všech běhů Ag v Env .

Funkční ekvivalence agentů

Ag_1, Ag_2 jsou **funkčně ekvivalentní vzhledem k** $Env \iff R(Ag_1, Env) = R(Ag_2, Env)$.

Prostě funkčně ekvivalentní (*simply functionally equivalent*) — funkčně ekvivalentní vzhledem ke *všem* prostředím.

Nástroj pro srovnání *vyjadřovací síly* architektur: implementace nepodstatná, vedou-li ke stejným pozorovatelným běhům.

1.6 Čistě reaktivní agent a agent s vnitřním stavem

Čistě reaktivní agent (*purely reactive / tropistic agent*)

$$Ag : E \rightarrow A$$

Reaguje jen na aktuální stav prostředí, historii ignoruje.

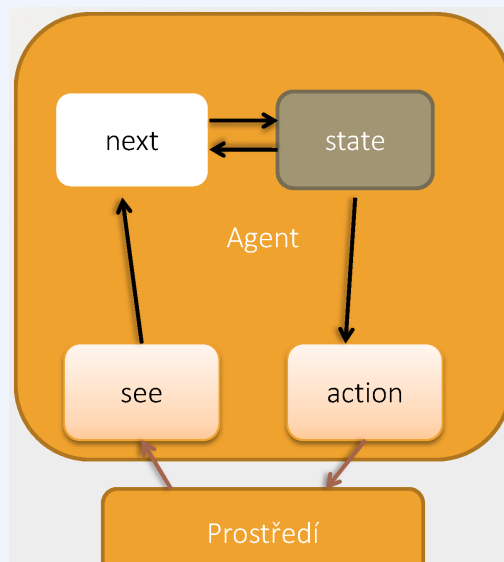
Vztah k obecnému agentovi: ke každému čistě reaktivnímu agentovi existuje funkčně ekvivalentní standardní agent; opačně *neplatí*.

Termostat: $Ag(e) = \text{off}$ pro $e = \text{„teplota OK“}$, jinak $Ag(e) = \text{on}$.

Agent s vnitřním stavem

I — vnitřní stavy agenta, Per — vjemy. Agent = trojice funkcí:

$see : E \rightarrow Per$	(vnímání)
$next : I \times Per \rightarrow I$	(aktualizace vnitřního stavu)
$action : I \rightarrow A$	(výběr akce)



Agent s vnitřním stavem za práce: start ve stavu i_0 , see převede stav prostředí na vjem, next z vjemu a starého stavu spočítá nový stav, action z něj vybere akci — ta zpětně mění prostředí.

Věta (bez důkazu): ke každému agentovi s vnitřním stavem existuje funkčně ekvivalentní standardní agent. Vnitřní stav *nezvyšuje vyjadřovací sílu*, je zásadní pro *efektivitu* a *přehlednost* implementace.

see formalizuje **pozorovatelnost**: $e_1 \neq e_2$ se stejným vjemem = agent je nerozliší (částečná pozorovatelnost). Krajní případy: $|Per| = |E|$, see prostá (plně pozorovatelné) vs. $|Per| = 1$ (agent nevidí nic).

1.7 Utilita: jak agentovi říct, co má dělat

Nezadrátované chování: agentovi říkáme *co*, ne *jak*. Standardní postup UI: cíl *nepřímo*, mírou úspěšnosti.

Utilita

Utilita stavu: $u : E \rightarrow \mathbb{R}$; cíl agenta = stavy s vysokou utilitou.

Utilita stavů je krátkozraká $\Rightarrow$ **utilita běhu**

$$u : R \rightarrow \mathbb{R},$$

pro dlouhodobě běžící agenty. Ze stavů např. utilita *nejhoršího* navštíveného stavu nebo *průměr* navštívených stavů — podle úlohy.

Optimální agent

$P(r \mid Ag, Env)$ — pravděpodobnost běhu r agenta Ag v Env , $\sum_{r \in R(Ag, Env)} P(r \mid Ag, Env) = 1$. Optimální agent maximalizuje *očekávanou utilitu*:

$$Ag_{opt} = \arg \max_{Ag \in AG} \sum_{r \in R(Ag, Env)} u(r) P(r \mid Ag, Env).$$

Elegantní, ale **nekonstruktivní** — neříká, jak agenta navrhnout; utilitu bývá těžké zadat. (Peníze nejsou dobrá utilita člověka: nelineární, jiná pro bohaté a chudé.)

Příklad: Tileworld — klasické testovací prostředí architektury. Šachovnice, pohyb do čtyř směrů; díry, překážky, bloky; cíl: zatlačit bloky do děr. *Dynamické*: díry, překážky i bloky náhodně vznikají

a zanikají. Utilita běhu

$$u(r) = \frac{N_f}{N_a},$$

N_f = díry zaplněné agentem během r , N_a = všechny díry vzniklé během r . Nutné *reagovat* (tlačný blok zmizel) i *využívat příležitosti* (vedle vznikl nový blok). Historicky nástroj pro kompromis reaktivita vs. proaktivita (odst. 2.5).

1.8 Predikátová specifikace úlohy

Utilita do $\{0, 1\}$: běh úspěšný ($u(r) = 1$), nebo ne.

Prostředí úlohy (*task environment*)

Predikátová specifikace $\Psi : R \rightarrow \{0, 1\}$: $\Psi(r)$ platí $\iff u(r) = 1$. **Prostředí úlohy** — dvojice $\langle Env, \Psi \rangle$: vlastnosti systému (přes Env) + kritérium splnění úlohy (přes Ψ).
 $R_\Psi(Ag, Env) = \{r \in R(Ag, Env) \mid \Psi(r)\}$.

Kdy agent úlohu řeší — tři výklady:

- **Pesimistický:** $R_\Psi(Ag, Env) = R(Ag, Env)$ — všechny běhy splňují Ψ .
- **Optimistický:** $\exists r \in R(Ag, Env) : \Psi(r)$ — alespoň jeden.
- **Realistický:** τ rozšířena o rozdělení pravděpodobnosti, úspěch měřen

$$P(\Psi \mid Ag, Env) = \sum_{r \in R_\Psi(Ag, Env)} P(r \mid Ag, Env).$$

Typy úloh

Úlohy dosažení (*achievement tasks*) — „něco najít“: cílové stavy $G \subseteq E$; $\Psi(r)$ platí, obsahuje-li r stav z G . Typicky klasické úlohy UI (prohledávání).

Úlohy udržení (*maintenance tasks*) — „něčemu se vyhnout“: „špatné“ stavy $B \subseteq E$; $\Psi(r)$ neplatí, obsahuje-li r stav z B . Typicky hry: B = prohrané pozice, prostředí = soupeř.

Kombinace: dosáhnout G a vyhnout se B .

1.9 Mechanismus výběru akcí — přehled

Mechanismus výběru akcí (*action selection mechanism*)

Procedura volby akce v každém okamžiku podle vjemů a vnitřního stavu. **Architektura agenta** — softwarová architektura realizující mechanismus; Maes (1991): „konkrétní metodologie stavby agentů, která určuje, jak lze agenta rozložit na množinu komponent a jak mají tyto komponenty interagovat, aby dohromady odpověděly na otázku, jak data ze senzorů a aktuální vnitřní stav určují akce a budoucí vnitřní stav agenta.“

Výběr akce = **klíčový problém návrhu agenta**; kapitola 2 = přehled realizací:

Rodina	Mechanismus výběru akce	Typický zástupce
symbolická / deliberativní	logická dedukce nad databází formulí; plánování	deduktivní agent, STRIPS, BDI, PRS
reaktivní (symbolická)	pravidla „situace $\rightarrow$ akce“ s pevnou prioritou	subsumpční architektura (Brooks)
reaktivní (konekcio-nistická)	šíření aktivace v síti modulů, vyhrává nejaktivnější	agent network architecture (Maes)
hybridní	vrstvy různé abstrakce + arbitráž mezi nimi	TouringMachines, InteRRaP
koaliční	soutěž dynamicky vznikajících koalic procesů o „vědomí“	IDA / globální pracovní prostor
naučená	maximalizace naučené Q -funkce / stochastická politika	Q-learning, actor-critic (kap. 6)

1.10 Shrnutí ke zkoušce

- Agent = autonomní systém zasazený do prostředí, vnímá a jedná, plní delegované cíle. MAS = interagující agenti *bez nutně sdíleného cíle*.
- Tři vlastnosti: reaktivita, proaktivita, sociálnost; těžké je vyvážení.
- Dennett: intencionální systém, tři postoje (fyzikální, návrhový, intencionální); intencionální postoj = abstrakce pro zvládnutí složitosti.
- Prostředí: 4 dichotomie (pozorovatelnost, statické/dynamické, determinismus, diskretnost); reálná prostředí v nejtěžší variantě.
- Formalismus: E, A , běh $r = e_0 a_0 e_1 \dots e_n$, R^A/R^E , $\tau : R^A \rightarrow 2^E$ (historie + nedeterminismus), $Env = \langle E, e_0, \tau \rangle$, $Ag : R^E \rightarrow A$, systém $\langle Ag, Env \rangle$.
- Funkční ekvivalence (vzhledem k prostředí / prostá). Čistě reaktivní $Ag : E \rightarrow A$ (slabší); agent s vnitřním stavem (*see, next, action*) — oba převeditelné na standardního agenta.
- Utilita $u : E \rightarrow \mathbb{R}$ vs. $u : R \rightarrow \mathbb{R}$; optimální agent maximalizuje *očekávanou* utilitu — nekonstruktivní. Tileworld, $u(r) = N_f/N_a$.
- Prostředí úlohy $\langle Env, \Psi \rangle$; pesimista/optimista/realista; úlohy dosažení (G) a udržení (B).
- **Mechanismus výběru akcí** = jádro otázky: vyjmenovat rodiny mechanismů a zástupce (tabulka výše).

2 Metody pro řízení agentů: symbolické a konekcionistické reaktivní plánování, hybridní přístupy

Jádro okruhu: konkrétní realizace mechanismu výběru akcí z odst. 1.9, od symbolického (deliberativního) pólu přes reaktivní k hybridním architektuám:

Přístup	Kde je v této kapitole
symbolické (deliberativní) řízení	deduktivní agent, BDI, STRIPS, PRS (odst. 2.1–2.6)
symbolické reaktivní plánování	subsumpční architektura (Brooks), odst. 2.8
konekcionistické reaktivní plánování	agent network architecture (Maes), odst. 2.9
hybridní přístupy	vrstvené architektury, TouringMachines, InteRRaP, IDA, odst. 2.11 a dále

2.1 Klasický přístup UI a deduktivní agent

Dva pilíře klasické UI:

1. **Symbolická reprezentace** prostředí a chování — logické formule.
2. **Syntaktická manipulace** s ní — dedukce, dokazování vět.

Teorie chování agenta = program (*spustitelná specifikace*). Dva klasické problémy:

- **Problém transdukce** (*transduction problem*) — převod reálného světa na symboly (počítačové vidění, zpracování přirozeného jazyka, učení).
- **Problém reprezentace a uvažování** — jak znalosti reprezentovat a efektivně nad nimi odvozovat (dokazovače, plánovače).

Deduktivní agent (agent jako dokazovač vět)

L — formule predikátové logiky prvního řádu, $D = 2^L$ — databáze formulí; vnitřní stav agenta $DB \in D$. Uvažování = odvozovací pravidla ρ :

$$DB \vdash_{\rho} \varphi \quad \text{— formulí } \varphi \text{ lze odvodit z } DB \text{ pomocí pravidel } \rho.$$

Agent = trojice $see : E \rightarrow Per$, $next : D \times Per \rightarrow D$, $action : D \rightarrow A$; $action$ dána procedurou níže.

Výběr akce jako dokazování.

```

1: function ACTION( $DB : D$ ) :  $A$ 
2:   for all  $a \in A$  do
3:     if  $DB \vdash_{\rho} Do(a)$  then return  $a$ 
4:     end if
5:   end for
6:   for all  $a \in A$  do
7:     if  $DB \not\vdash_{\rho} \neg Do(a)$  then return  $a$ 
8:     end if
9:   end for
10:  return null
11: end function

```

1. cyklus: akce *dokazatelně* žádoucí ($Do(a)$); 2. cyklus: akce aspoň *konzistentní* s DB (nelze dokázat $\neg Do(a)$); jinak nic.

Příklad: vysavačový svět 3×3 . Predikáty $In(x, y)$ (pozice agenta), $Dirt(x, y)$ (špína), $Facing(d)$ (směr). Pravidla:

$$\begin{aligned}
& In(x, y) \wedge Dirt(x, y) \Rightarrow Do(suck) \\
& In(0, 0) \wedge Facing(north) \wedge \neg Dirt(0, 0) \Rightarrow Do(forward) \\
& In(0, 2) \wedge Facing(north) \wedge \neg Dirt(0, 2) \Rightarrow Do(turn) \quad \dots
\end{aligned}$$

Agent projde svět „hadem“ 00–01–02–12–11–10–...; úklid má přednost před pohybem (pravidlo *suck* první, ostatní mají v předpokladu $\neg Dirt$).

Klady a zápory.

- + Elegantní, jasná sémantika; specifikace = program.
- Prakticky nepoužitelné pro netriviální úlohy: dokazování pomalé, nemusí skončit.
- **Problém vypočitatelné racionality** (*calculative rationality*): akce optimální pro stav prostředí *na začátku* odvozování — mezitím se změnilo. Katastrofa v rychle se měnícím prostředí.
- Těžký návrh *see* (obrázek $\rightarrow$ formule, reprezentace času).

Prvky přístupu přežívají v pozdějších architekturách, zejména BDI.

2.2 Praktické uvažování: architektura BDI

Praktické uvažování (*practical reasoning*)

Inspirace lidským rozhodováním. *Teoretické* uvažování (Sókratés) → co si o světě myslíme; *praktické* → **akce**. Dvě fáze:

- **Deliberace** (*deliberation*) — čeho chceme dosáhnout („dodělat magisterské studium“).
- **Uvažování o prostředcích** (*means-ends reasoning*) = plánování — *jak* (plán, jak dostudovat).

Obojí nesmí trvat příliš dlouho.

Beliefs — Desires — Intentions

Beliefs (přesvědčení) — informační stav, znalosti o světě. Ne „znalosti“: jsou *subjektivní*, *nemusí být pravdivé*, *mohou se změnit*. Mohou obsahovat i odvozovací pravidla (dopředné řetězení).

Desires (touhy) — motivační stav, cíle/situace k dosažení. *Mohou být vzájemně nekonzistentní* (učit se i jít na pivo).

Intentions (záměry) — stavy světa, k jejichž dosažení se agent *zavázal*; vedou k akcím (agent kvůli nim plánuje). **Přetrvávají**, dokud

1. jich agent nedosáhne, nebo
2. nezačne věřit, že jsou nedosažitelné, nebo
3. nepominou důvody, které k nim vedly.

Dále *omezují další deliberaci* (nezvažují možnosti neslučitelné se záměrem) a *ovlivňují budoucí přesvědčení* (počítám s úspěchem záměru).

Touha vs. záměr (Bratman, 1990): touha (zahrát si odpoledne basketbal) = potenciální vliv soupeřící s ostatními touhami; záměr = rozhodnuto, pro a proti už nevažuji.

Funkce deliberace

$B \subseteq Bel$, $D \subseteq Des$, $I \subseteq Int$ — aktuální přesvědčení, touhy, záměry. Tři funkce:

- $brf : 2^{Bel} \times Per \rightarrow 2^{Bel}$ — *belief revision function*, aktualizace přesvědčení podle vjemu;
- $options : 2^{Bel} \times 2^{Int} \rightarrow 2^{Des}$ — generování možností (tužeb) z přesvědčení a stávajících záměrů;
- $filter : 2^{Bel} \times 2^{Des} \times 2^{Int} \rightarrow 2^{Int}$ — výběr záměrů z možností.

Klíčové rozdělení: *generování* levné a široké, *filtrování* = vlastní rozhodování (bere v úvahu stávající závazky).

2.3 Plánování: STRIPS

Uvažování o prostředcích / plánování

Jak dosáhnout cíle (záměru) dostupnými prostředky (akcemi).

- **Vstup:** cíl (= záměr I); stav prostředí (= přesvědčení B); akce Ac .
- **Výstup:** plán = posloupnost akcí, po níž je cíl splněn.

STRIPS (Fikes, Nilsson, 1971): svět = množina formulí FOL; akce = *schéma* s předpoklady a efekty (přidávané/mazané fakty). Algoritmus: rozdíl cíl vs. stav, vhodná akce ho zmenší (*means-ends analysis*). Hezké, málo praktické — utápí se v detailech nízké úrovně.

Deskriptor akce, plánovací problém, plán

Deskriptor akce a : trojice $\langle P_a, D_a, A_a \rangle$ — předpoklady (*preconditions*), mazaná množina (*delete list*), přidávaná množina (*add list*); uzemněné atomické formule (bez spojek a proměnných).

Plánovací problém: $\langle B_0, O, G \rangle$ — počáteční přesvědčení, deskriptory všech akcí $O = \{ \langle P_a, D_a, A_a \rangle : a \in Ac \}$, cíl G (množina formulí).

Plán $\pi = (a_1, \dots, a_n)$, $a_i \in Ac$, určuje databáze

$$B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}, \quad i = 1, \dots, n.$$

Přípustný (*admissible*): $B_{i-1} \models P_{a_i}$ pro každé i . **Korektní/validní** (*correct*): přípustný a $B_n \models G$.

Plánování = pro $\langle B_0, O, G \rangle$ najít korektní plán, nebo prohlásit, že neexistuje.

Příklad: svět kostek (*blocks world*). Predikáty: $On(x, y)$ (x na y), $OnTable(x)$, $Clear(x)$ (na x nic), $Holding(x)$ (rameno drží x), $ArmEmpty$. Akce:

Akce	Pre / Del	Add
$Stack(x, y)$	Pre: $\{Clear(y), Holding(x)\}$ Del: $\{Clear(y), Holding(x)\}$	$\{ArmEmpty, On(x, y)\}$
$UnStack(x, y)$	Pre: $\{On(x, y), Clear(x), ArmEmpty\}$ Del: $\{On(x, y), ArmEmpty\}$	$\{Holding(x), Clear(y)\}$
$Pickup(x)$	Pre: $\{OnTable(x), Clear(x), ArmEmpty\}$ Del: $\{OnTable(x), ArmEmpty\}$	$\{Holding(x)\}$
$PutDown(x)$	Pre: $\{Holding(x)\}$ Del: $\{Holding(x)\}$	$\{ArmEmpty, OnTable(x)\}$

$$B_0 = \{Clear(A), On(A, B), OnTable(B), OnTable(C), Clear(C), ArmEmpty\},$$

$$G = \{OnTable(A), OnTable(B), OnTable(C)\}.$$

Korektní plán $\pi = (UnStack(A, B), PutDown(A))$: $B_0 \models \text{Pre 1. akce}$;

$$B_1 = (B_0 \setminus \{On(A, B), ArmEmpty\}) \cup \{Holding(A), Clear(B)\} \models Holding(A);$$

$B_2 \models G \Rightarrow$ přípustný i korektní.

Pomocné funkce nad plány

$pre(\pi)$ (předpoklad), $body(\pi)$ (tělo), $empty(\pi)$, $execute(\pi)$, $hd(\pi)$ (první akce), $tail(\pi)$ (zbytek), $sound(\pi, I, B)$ (korektnost vzhledem k záměrům I a přesvědčením B); plánovací funkce $plan : 2^{Bel} \times 2^{Int} \times 2^{Ac} \rightarrow Plan$.

Knihovna plánů. Online konstrukce plánů je drahá; $plan()$ často = *knihovna hotových plánů*: projít, zkontrolovat předpoklady vs. přesvědčení a efekty vs. cíl. Jádro PRS (sekce 2.6).

2.4 Implementace BDI agenta

Hlavní řídicí smyčka BDI agenta:

- 1: $B \leftarrow B_0; \quad I \leftarrow I_0$
- 2: **while true do**
- 3: $v \leftarrow see()$
- 4: $B \leftarrow brf(B, v)$
- 5: $D \leftarrow options(B, I)$
- 6: $I \leftarrow filter(B, D, I)$
- 7: $\pi \leftarrow plan(B, I, Ac)$
- 8: **while** $\neg(empty(\pi) \vee succeeded(I, B) \vee impossible(I, B))$ **do**
- 9: $a \leftarrow hd(\pi); \quad execute(a); \quad \pi \leftarrow tail(\pi)$
- 10: $v \leftarrow see(); \quad B \leftarrow brf(B, v)$
- 11: **if reconsider(I, B) then**
- 12: $D \leftarrow options(B, I); \quad I \leftarrow filter(B, D, I)$

```

13:      end if
14:      if  $\neg sound(\pi, I, B)$  then
15:           $\pi \leftarrow plan(B, I, Ac)$  ▷ přeplánování
16:      end if
17:  end while
18: end while

```

$succeeded(I, B)$ — záměry I platí za přesvědčení B ; $impossible(I, B)$ — nemohou platit. Vnitřní cyklus končí: plán vyčerpán, cíl splněn, nebo nedosažitelný.

2.5 Odhodlání agenta (*commitment*)

Strategie odhodlání vůči záměrům

Kdy opustit cíl?

- **Slepé odhodlání** (*blind commitment*, též *fanatical*) — dokud *nezačne věřit*, že ho *dosáhl*. (Nikdy se nevzdá.)
- **Jednosměrné odhodlání** (*single-minded commitment*) — dokud *nezačne věřit*, že ho *dosáhl*, *nebo* že už není dosažitelný.
- **Otevřené odhodlání** (*open-minded commitment*) — dokud věří, že je dosažitelný (a stále chtěný) — lze opustit i při změně motivace.

Odhodlání vůči plánu vs. vůči cíli: algoritmus výše je odhodlán plánu — opustí ho při splnění cíle, nedosažitelnosti, nebo prázdném plánu. Těžší: kdy *přehodnotit sám záměr*? Dilema — deliberace **trvá čas**, prostředí se mění:

- *nepřehodnocuje* → upne se k nedosažitelným cílům;
- *přehodnocuje příliš často* → samé uvažování, nic nestihne.

Metařízení (Wooldridge, Parsons, 1999): *reconsider()* výpočetně *jednodušší* než samo přehodnocení. Kvalita: *dobrá* $\iff$ kdykoli navrhne přehodnocení, agent po deliberaci záměr *skutečně* změní (a naopak).

Odvážný vs. opatrný agent

Odvážný agent (*bold agent*) — přehodnocuje až po dokončení celého plánu (nikdy plán nepřeruší kvůli deliberaci).

Opatrný agent (*cautious agent*) — přehodnocuje po každé akci plánu.

Míra odvahy (*level of boldness*) — počet akcí mezi dvěma deliberacemi.

Dynamičnost prostředí (*rate of world change*) — počet změn prostředí za jeden cyklus agenta.

Efektivita agenta = dosažené záměry / všechny záměry.

Experiment (Kinny, Georgeff, Tileworld):

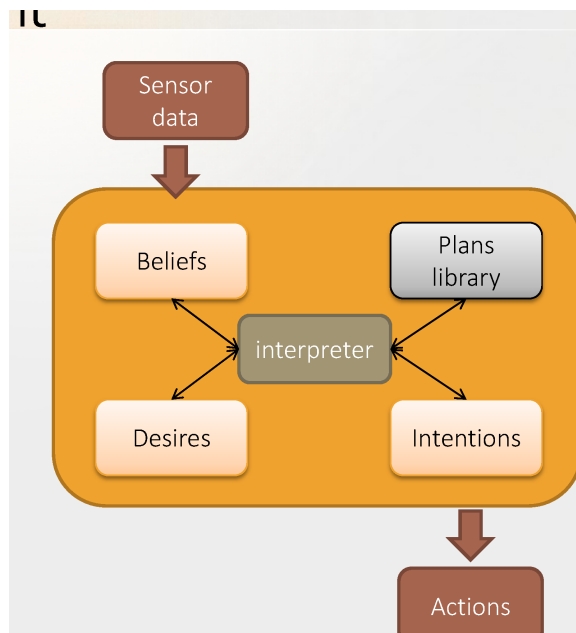
- svět se mění *pomalu* → efektivnější **odvážní** (deliberace = zbytečná režie);
- *rychle* → **opatrní** (plány rychle zastarávají).

Kvantitativní podoba kompromisu reaktivita vs. proaktivita.

2.6 Procedurální odvozovací systém (PRS)

PRS — *Procedural Reasoning System*

Georgeff a kol., 80. léta, Stanford. **První implementace BDI**, jedna z prakticky nejúspěšnějších architektur; reimplementace: **AgentSpeak/Jason**, **JAM**, **JACK**, **Jadex**. Nasazení: OASIS (řízení letového provozu v Sydney), SPOC, SWARM.



Architektura PRS agenta. Data ze senzorů vstupují do databáze beliefs (atomy FOL prologovského typu); interpret propojuje beliefs, desires (cíle), knihovnu plánů a zásobník intentions a vydává akce.

Plán v PRS

Knihovna hotových plánů = *procedurální znalosti* agenta. **Neplánuje se od nuly**, jen se *vybírá*.
Tři části plánu:

- **Goal** (*invocation condition / cue*) — co má platit po vykonání; na jaký cíl se plán hodí.
- **Context** — podmínka spuštění (konzistentní s přesvědčeními).
- **Body** — akce k vykonání.

Tělo *není* jen lineární posloupnost akcí — může obsahovat *cíle*: „dosáhni *f*“, „dosáhni *f* nebo *g*“, „dosahuj *f*, dokud nenastane *g*“ ⇒ hierarchický rozklad cílů na podcíle.

Běh interpretu:

1. Inicializace: B_0 a cíl nejvyšší úrovně (*top-level goal*).
2. Zásobník záměrů: aktuální cíle v různém stavu rozpracovanosti.
3. Záměr z vrcholu zásobníku → v knihovně hledá plány s odpovídajícím cílem.
4. Z nich ty s *kontextem* konzistentním s přesvědčeními = aktuální *options / desires*.
5. **Deliberace** = výběr jednoho záměru:
 - původní PRS: *metaplány* (plány o plánech modifikující záměry) — příliš složité;
 - praktičtější: číselné *ohodnocení* plánu (očekávaná utilita), vybrat nejvyšší.
6. Vykonání plánu → může přidat další záměry na zásobník.
7. Selhání plánu → jiný záměr z možností.

Ukázka — Jason/AgentSpeak (agent nosí majiteli pivo z ledničky):

```

available(beer,fridge). limit(beer,10). /* počáteční beliefs */
too_much(B) :- .count(consumed(_,_ ,B),Q) & limit(B,L) & Q > L. /* pravidlo */
+!has(owner,beer) /* spouštěcí událost */
: available(beer,fridge) & not too_much(beer) /* kontext */
<- !at(robot,fridge); open(fridge); get(beer); close(fridge); /* tělo */
!at(robot,owner); hand_in(beer).
+!at(robot,P) : not at(robot,P) <- move_towards(P); !at(robot,P).

```

+!g : c <- b = „při události přidání cíle *g* a platném kontextu *c* vykonej tělo *b*“. ! — *achievement goal* (dosáhni); +/- — přidání/odebrání přesvědčení nebo cíle; .send, .count — vestavěné akce.

2.7 Reaktivní přístup a jeho východiska

80. a 90. léta: drobné úpravy symbolického přístupu nestačí → alternativní paradigmaty UI. Společný jmenovatel: **odmítnutí symbolické reprezentace** a uvažování syntaktickou manipulací. Tři teze:

Tři principy reaktivního přístupu

Interakce (*situatedness*) — inteligentní chování závisí na prostředí, do něhož je agent *zasazen*; nelze studovat izolovaně.

Ztělesnění (*embodiment*) — není jen logika; produkt agenta, který *má tělo* a fyzicky interaguje se světem.

Emergence — inteligentní chování *vzniká* interakcí mnoha jednoduchých chování; nikde explicitně naprogramováno.

Charakteristiky reaktivních agentů: *behaviorální* (vývoj a kombinace jednotlivých chování), *situovaní*, *reaktivní* (jen reagují, nedeliberují). Reprezentace **subsymbolická**: konekcionistické sítě, konečné automaty, pravidla IF–THEN.

2.8 Symbolické reaktivní plánování: subsumpční architektura

Subsumpční architektura (*subsumption architecture*)

Rodney Brooks, 1986–1991; nejúspěšnější reaktivní přístup. Tři Brooksovy teze:

1. Inteligentní chování *bez symbolické reprezentace*.
2. Inteligentní chování *bez explicitního abstraktního uvažování*.
3. Intelligence = *emergentní vlastnost* určitých složitých systémů.

Intelligence agenta = množina jednoduchých cílově orientovaných **chování** (*behaviours*):

- každé chování = mechanismus výběru akce (dvojice *situace* → *akce*): vjemy → akce, jeden dílčí cíl, struktura jednoduchého pravidla;
- běží *paralelně*, *soupeří* o kontrolu nad agentem;
- uspořádána do **subsumpční hierarchie** = priority (vyšší vrstva „pohlcuje“ = potlačuje nižší).

Mechanismus výběru akce v subsumpční architektuře:

- ```
1: function ACTION(vjem p)
2: fired ← { b ∈ Behaviours | p splňuje podmínku b } ▷ chování, která „vystřelí“
3: for all b ∈ fired do
4: if neexistuje b' ∈ fired takové, že b' < b then ▷ b není nikým inhibováno
5: return akce příslušná chování b
6: end if
7: end for
8: return null ▷ nevystřelilo nic, agent nedělá nic
9: end function
```

Formálně: chování = dvojice  $\langle c, a \rangle$ ,  $c \subseteq Per$  podmínka,  $a \in A$  akce; na množině chování  $R$  relace inhibice  $\prec \subseteq R \times R$  (striktní totální uspořádání). **Pozor na směr:** u Wooldridge i na přednášce  $b_1 \prec b_2 = b_1$  **inhibuje**  $b_2$  —  $b_1$  je *níže* v hierarchii a má *přednost*; zápis  $b_1 \prec b_2 \prec \dots \prec b_n$  = od nejvyšší priority k nejnižší.

**Vlastnosti:** *jednoduchý* (při desítkách chování záludné priority), *velmi rychlý* — hardwarová implementace, konstantní složitost.

**Příklad: Steelsův průzkumník Marsu.** Úloha: roj robotů sbírá vzácné vzorky hornin na cizí planetě. Prostředky: navigační signál základny (robot detekuje jen *gradient*); radioaktivní droby = *nepřímá komunikace* (stigmergie); bez přímé komunikace.

*První iterace* — náhodná procházka a návrat jednoho robota (priorita klesá shora dolů):



---

```

R1 if vidím překážku then změň směr
R2 if nesu vzorek and jsem na základně then polož vzorek
R3 if nesu vzorek and nejsem na základně then jdi po gradientu nahoru
R4 if vidím vzorek then seber ho
R5 if true then pohybuj se náhodně

```

---

*Druhá iterace* — průzkum se stigmergií: vzorky bývají ve shlucích; nálezce cestou domů rozhazuje drobký = „řekne“ to ostatním:

---

```

R6 if nesu vzorek and jsem na základně then polož vzorek
R7 if nesu vzorek and nejsem na základně then polož 2 drobký a jdi po gradientu nahoru
R8 if cítím drobek then seber 1 drobek a jdi po gradientu dolů

```

---

Priority („levé inhibuje pravé“, od nejvyšší):  $R1 \prec R6 \prec R7 \prec R4 \prec R8 \prec R5$ . Robot na stopě bere po jednom drobký  $\rightarrow$  stopa se po vytěžení shluku *vypaří*. Kooperace roje bez modelu světa a bez zpráv.

## 2.9 Konekcionistické reaktivní plánování: agent network architecture

Subsumpce: konflikty řeší *pevná* priorita (rigidní). Konekcionistický přístup: **spojitá aktivace šířená sítí** — výběr akce = dynamika sítě, ne pevné pořadí. „Reaktivní plánování“: agent se chová, jako by plánoval, explicitní plán nikdy nevytvoří.

### Agent network architecture / spreading activation networks

Pattie Maes, 1989–1991. Agent = množina **kompetenčních modulů** (*competence modules*), obdoba Brooksových chování. Modul  $i$  má:

- **předpoklady**  $c_i$  (*preconditions*) — podmínka *proveditelnosti* (*executable*);
- **efekty**: *add list*  $a_i$ , *delete list*  $d_i$  (jako u STRIPS);
- **úroveň aktivace**  $\alpha_i$ ; globální **práh aktivace**  $\theta$ .

Moduly = orientovaný graf: hrany ze shody pre- a postpodmínek + hrany časové následnosti a konfliktů. Každý krok: šíření aktivace; **vybrán proveditelný modul s nejvyšší aktivací nad prahem  $\theta$** .

### Tři zdroje aktivace:

- **stav světa** (*state*) — moduly se splněnými předpoklady („teď se dá udělat tohle“);
- **cíle** (*goals*) — moduly s cílem v *add listu* („tohle vede k cíli“);
- **ochranné cíle** (*protected goals*) — moduly se splněným cílem v *delete listu* jsou *inhibovány* („tohle by zbořilo, co už mám“).

### Tři typy spojů mezi moduly:

- **následnické** (*successor links*):  $x \rightarrow y$ , pokud  $y$  přidává předpoklad  $x$  — *dopředné* řetězení („až budu hotov, půjde to dál“);
- **předchůdcovské** (*predecessor links*): neproveditelný  $x$  aktivuje moduly, které mu splní chybějící předpoklad — *zpětné* řetězení („pomoz mi, ať můžu běžet“);
- **konfliktní** (*conflict links*):  $x$  *inhibuje*  $y$ , které ruší jeho předpoklady.

### Pět globálních parametrů:

- $\theta$  — práh aktivace (jediný v systému) = **uvážlivost vs. rychlost** (vysoký práh: déle váhá, lépe rozhoduje);
- $\phi$  — aktivace vpravená stavem světa;  $\gamma$  — aktivace vpravená cíli; poměr  $\gamma/\phi$  = **proaktivita vs. reaktivita**;
- $\delta$  — aktivace odebraná chráněnými cíli;
- $\pi$  — *průměrná* aktivace, na kterou se síť po každém kroku normalizuje (celková energie konstantní).



Žádný proveditelný modul nad prahem  $\rightarrow \theta$  klesne o 10 %, znovu — síť nemůže „zamrznout“.

**Jeden krok výpočtu aktivace:** (1) každý pravdivý fakt rozdělí  $\phi$  modulům, které ho mají v předpokladech; (2) každý cíl rozdělí  $\gamma$  modulům, které ho přidávají; chráněné cíle *odeberou*  $\delta$  modulům, které by je rušily; (3) proveditelné moduly posílají aktivaci následníkům, neproveditelné zpět modulům plnicím chybějící předpoklad, konfliktní spoje odečítají; (4) přeskálování na průměr  $\pi$ . Pak výběr proveditelného modulu nad  $\theta$ ; provedený modul aktivaci vynuluje.

*Poznámka k přednášce:* slidy uvádějí práh jako vlastnost modulu („priorita“); v Maesově původním modelu je  $\theta$  *globální* a „prioritou“ je dynamicky se měnící aktivace  $\alpha_i$ .

**Srovnání se subsumpcí:** pořadí chování ne pevné, ale *dynamické* ze stavu světa a cílů. Síť generuje i víceukrové „plánovitě“ sekvence (aktivace řetězená zpět od cíle předchůdcovskými spoji) bez explicitní datové struktury plánu — odtud *reaktivní plánování*. Cena: citlivost na parametry, bez záruky optimality a úplnosti.

## 2.10 Omezení reaktivních architektur

- **Bez modelu světa:** vše z aktuálních vjemů; paměť nutno *externalizovat* změnou prostředí (radioaktivní dročky) — stigmergie.
- **Krátkodobý pohled:** jen aktuální stav a lokální informace; globální podmínky a dlouhodobé cíle obtížné.
- **Emergence není inženýrský postup:** chování se ladí experimentálně, chybí metodologie i verifikace.
- **Škálování:** několik vrstev zvládnutelné, **desítky chování neřešitelné** (interakce rostou kvadraticky).

## 2.11 Hybridní architektury

Čistě reaktivní ani čistě deliberativní architektura není ideální  $\rightarrow$  kombinace:

- **deliberativní / plánovací komponenta** — symbolická úroveň, reprezentace, plány;
- **reaktivní komponenta** — okamžité reakce bez složitých výpočtů.

Uspořádání do **vrstev** (*layered architectures*); reaktivní vrstvy mají zpravidla *přednost* (bezpečnost před optimalitou).

### Horizontální vs. vertikální vrstvení

**Horizontální vrstvení:** všechny vrstvy *paralelně* připojeny k senzorům i efektorům; každá vrstva = samostatný agent navrhuje akci.

+ Koncepčně jednoduché:  $n$  druhů chování =  $n$  vrstev.

– Návrhy vrstev si odporují  $\rightarrow$  nutná **mediační funkce** (*mediator function*); úzké hrdlo:  $m^n$  kombinací ( $n$  vrstev,  $m$  návrhů), kritický bod selhání.

**Vertikální vrstvení:** vrstvy zapojeny *sériově*.

- *One-pass:* vjem do nejnižší vrstvy, průchod nahoru, akce z nejvyšší; přirozená hierarchie chování.

- *Two-pass:* vjemy *zdola nahoru*, řídicí povely (akce) *shora dolů*; jako řízení ve firmě.

+ Bez mediace, lineární složitost interakcí ( $n - 1$  rozhraní).

– Křehké: selhání kterékoli vrstvy shodí celého agenta.

### TouringMachines (Ferguson, 1992)

**Horizontální třívrstvá architektura;** simulovaný agent v dopravním prostředí.

- **Modelovací vrstva** (*modelling layer*) — symbolické modely agenta, prostředí a ostatních agentů; předvídá a řeší konflikty, stanovuje cíle pro plánovací vrstvu.
- **Plánovací vrstva** (*planning layer*) — proaktivní chování; výběr z předpřipravených plánů

(jako PRS), sestavení plánu cesty.

- **Reaktivní vrstva** (*reactive layer*) — pravidla *situace* → *akce*, okamžitá reakce (vyhýbání překážkám).

Vrstvy běží paralelně; výstupy sjednocují **řídící pravidla** (*control rules*) = konkrétní mediační funkce; potlačují vstupy a výstupy vrstev (*censor* a *suppressor* pravidla).

### InteRRaP (Müller, 1994)

**Vertikální dvoupřechodová** (*two-pass*) třívrstvá architektura.

- **Vrstva kooperativního plánování** (*cooperative planning layer*) — sociální interakce, společné plány s ostatními agenty.
- **Vrstva lokálního plánování** (*local planning layer*) — individuální plánování.
- **Behaviorální vrstva** (*behaviour-based layer*) — reaktivní chování.

Každá vrstva má *vlastní znalostní bázi* své úrovně abstrakce (svět / plány / společné plány a modely ostatních agentů). Dvě operace mezi vrstvami: *aktivace zdola nahoru* (nižší vrstva situaci nezvládne, předá výš), *zásah shora dolů* (vyšší vrstva vykonává svá rozhodnutí přes nižší).

**Reálný příklad: Stanley** (Stanford) — autonomní vůz, vítěz DARPA Grand Challenge 2005. Vrstvy: senzory → abstraktní percepce → plánování a řízení → rozhraní vozidla; navíc vrstvy uživatelského rozhraní a globálních služeb.

## 2.12 Komplexní architektury: IDA a globální pracovní prostor

Opak reaktivního minimalismu: integrovat do jednoho agenta *všechno* — výběr akce, paměť, deliberaci, emoce.

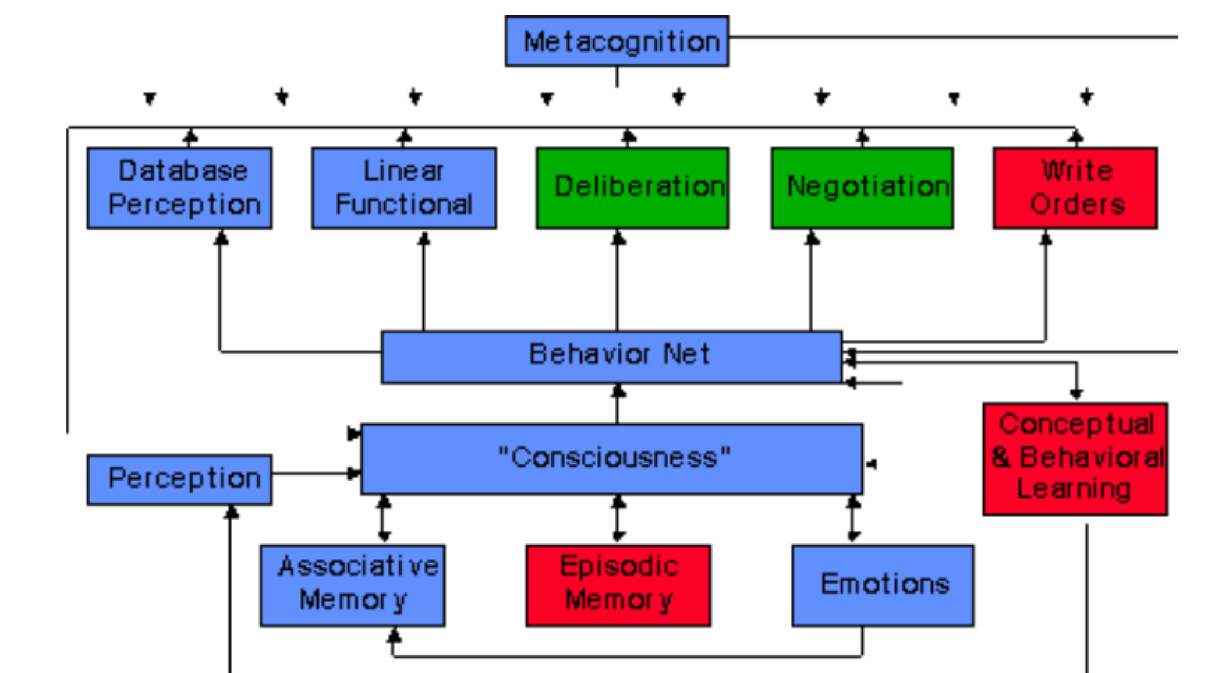
### IDA — *Intelligent Distribution Agent* (S. Franklin)

Jedna z nejsložitějších postavených agentních architektur. Reálná úloha amerického námořnictva: přidělování personálu na nová služební místa (odtud „distribution“); distribuovaná je i architektura sama.

**Východisko: teorie globálního pracovního prostoru** (*global workspace theory*, Baars, 1988–97) — jedna z teorií vědomí:

- mysl je MAS: mnoho jednoduchých, převážně nevědomých specializovaných procesů;
- komunikace *zřídka*, jen přes sdílenou paměť typu *blackboard* organizovanou jako asociativní pole (srov. kap. 5);
- procesy dynamicky tvoří **koalice** vyšších řádů;
- **výběr akce**: koalice nejlépe odpovídající aktuálním vstupům se dostane do „vědomí“ (globálního pracovního prostoru) a vykoná se;
- *hierarchie kontextů* — kontext cílů, vjemů, smyslů, konceptů, kulturní kontext.

**Hodnocení:** kombinuje mnoho přístupů z MAS i zbytku UI (výběr akce, paměť, deliberace, emoce) — zároveň slabina: řada rozhodnutí *ad hoc*, sladění heterogenních modelů do optimálně spolupracujícího celku velmi obtížné.



Architektura IDA. Modré bloky jsou moduly vnímání a paměti, zelené deliberace a vyjednávání, červené učení a výstup; vše se setkává v „Consciousness“ (globálním pracovním prostoru) a v Behavior Net, který vybírá akci.

IDA = čtvrtý typ arbitráže mezi chováními: subsumpce **pevná priorita**, síť Maes **úroveň aktivace**, vrstvené architektury **mediační funkce**, IDA **soutěž dynamicky vznikajících koalic** o přístup do globálního pracovního prostoru.

## 2.13 Srovnání architektur

|              | Deduktivní            | BDI / PRS                    | Reaktivní                | Hybridní            |
|--------------|-----------------------|------------------------------|--------------------------|---------------------|
| Reprezentace | symbolická (FOL)      | symbolická (beliefs, plány)  | subsymbolická / pravidla | smíšená po vrstvách |
| Výběr akce   | dokazování $Do(a)$    | deliberace + plán z knihovny | priorita / aktivace      | vrstvy + arbitráž   |
| Model světa  | úplný                 | částečný, revidovatelný      | žádný                    | po vrstvách         |
| Rychlost     | velmi nízká           | střední                      | vysoká (konst.)          | vysoká v reflexu    |
| Reaktivita   | špatná                | střední (laditelná)          | výborná                  | dobrá               |
| Proaktivita  | dobrá                 | výborná                      | žádná                    | dobrá               |
| Návrh        | obtížný, ale formální | systematický                 | experimentální           | složitý             |

## 2.14 Shrnutí ke zkoušce

- **Deduktivní agent:**  $DB \in 2^L$ ,  $DB \vdash_{\rho} Do(a)$ ; dvoufázový výběr akce (dokazatelná akce  $\rightarrow$  konzistentní akce  $\rightarrow$  nic). Elegantní, nepraktický; problém transdukce a vypočitatelné racionality.
- **BDI:** dvě fáze — deliberace (*brf*, *options*, *filter*) a means-ends reasoning (plánování). Beliefs subjektivní a omylné, desires mohou být nekonzistentní, intentions = závazky: přetrvávají, omezují další deliberaci.
- **STRIPS:** deskriptor akce  $\langle P_a, D_a, A_a \rangle$ , plánovací problém  $\langle B_0, O, G \rangle$ ,  $B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}$ ; plán *přípustný* ( $B_{i-1} \models P_{a_i}$ ) vs. *korektní* (navíc  $B_n \models G$ ). Umět projít svět kostek.
- **BDI smyčka** a role *reconsider* a *sound*.
- **Commitment:** blind / single-minded / open-minded; bold vs. cautious agent; efektivita = dosažené/všechny záměry; pomalé prostředí  $\rightarrow$  bold, rychlé  $\rightarrow$  cautious.
- **PRS:** první implementace BDI; plán = Goal + Context + Body; knihovna plánů místo plánování; zásobník záměrů; deliberace metaplány nebo utilitou; potomci AgentSpeak/Jason, JAM, JACK,

Jadex.

- Reaktivní přístup: trojice *situatedness* — *embodiment* — *emergence*; odmítá symbolickou reprezentaci.
- **Subsumpce** (Brooks) = *symbolické* reaktivní plánování: jednoduchá chování *situace* → *akce*, paralelní, pevná priorita. Umět Steelsova průzkumníka Marsu a roli stigmergie (drobky).
- **Agent network architecture** (Maes) = *konekcionalistické* reaktivní plánování: moduly s předpoklady a efekty (add, delete); aktivace se šíří následnickými, předchůdcovskými a konfliktními spoji; zdroje: stav světa, cíle, inhibice od chráněných cílů; vybere se proveditelný modul s nejvyšší aktivací nad prahem.  $\gamma/\phi$  ladí proaktivitu vs. reaktivitu.
- Omezení reaktivních agentů: bez modelu světa, krátkodobý pohled, neinženýrský návrh, neškálovatelné množství vrstev.
- **Hybridní**: horizontální (paralelní vrstvy + mediátor,  $m^n$  kombinací, úzké hrdlo) vs. vertikální (sériové, one-pass/two-pass,  $n - 1$  rozhraní, křehké).
- **TouringMachines** = horizontální 3 vrstvy (modelling, planning, reactive) + řídicí pravidla. **InteRRaP** = vertikální two-pass 3 vrstvy (kooperativní plánování, lokální plánování, behaviorální) + vlastní KB na vrstvu. **Stanley** = reálné nasazení.
- **IDA** (Franklin) + *global workspace theory* (Baars): mysl jako MAS jednoduchých nevědomých procesů komunikujících přes blackboard; koalice soutěží o vstup do „vědomí“, vítězná se vykoná; hierarchie kontextů. Nejsložitější architektura, mnoho ad hoc rozhodnutí.
- Čtyři typy arbitráže mezi chováními: **pevná priorita** (subsumpce), **úroveň aktivace** (Maes), **mediační funkce** (vrstvené), **soutěž koalic** (IDA).

### 3 Problém hledání cesty

♦ **nad rámec slidů: celá kapitola** NAIL106 plánování i robotiku výslovně *nepokrývá*, slidy k hledání cesty nemají nic; okruh ho ale jmenuje ⇒ nutné umět. Minimum, které na zkoušce obstojí: A\* s vlastnostmi heuristiky + *multiagentní* část — dynamické prostředí, koordinace více agentů.

#### 3.1 Formulace a zařazení

Hledání cesty (*pathfinding*) — nejčastější konkrétní podúloha situovaného agenta:

- kde: součást plánovací vrstvy hybridní architektury, akce `move_towards(P)` v BDI plánu, nebo služba poskytovaná jiným agentům;
- multiagentní specifika: prostředí **dynamické** (mapa se mění), ostatní agenti = **pohyblivé překážky**, nutná koordinace.

#### Problém hledání cesty

Ohodnocený graf  $G = (V, E)$ , nezáporné ohodnocení hran  $w : E \rightarrow \mathbb{R}_0^+$ , počáteční vrchol  $s \in V$ , cílový vrchol  $g \in V$ . Hledáme cestu  $s = v_0, v_1, \dots, v_k = g$  minimalizující  $\sum_i w(v_{i-1}, v_i)$ . Graf bývá zadán *implicitně* nástupnickou funkcí (mřížka 4-/8-souvislá, navigační síť, graf viditelnosti).

♦ **nad rámec slidů** **Metaheuristiky** — *mravenčí algoritmy* (ACO) a *optimalizace rojem částic* (PSO) z okruhu *Přírodou inspirované počítání*:

- ACO: inspirováno hledáním nejkratší cesty k potravě; směřování v sítích (AntNet), cesta robota v mřížce;
- PSO: v robotice cesta = vektor průjezdových bodů, spojitě optimalizována délka, hladkost, odstup od překážek;
- **pro zadání výše se nevyplatí**: statický graf se známými cenami ⇒ A\* *optimální* a rychlejší, metaheuristika bez záruky optimality;
- smysl až u úloh těžších než nejkratší cesta: spojitý prostor s překážkami, více kritérií, silně dynamické či stochastické ceny, směřování bez globální znalosti topologie.

### 3.2 A\*

Prohledávání typu *best-first*; pro každý vrchol  $n$ :

$g(n)$  = cena nejlepší dosud nalezené cesty  $s \rightarrow n$ ,

$h(n)$  = heuristický odhad ceny  $n \rightarrow g_{\text{cil}}$ ,

$f(n) = g(n) + h(n)$  — odhad ceny nejlepší cesty  $s \rightarrow g_{\text{cil}}$  vedoucí přes  $n$ .

**Algoritmus A\*** (Hart, Nilsson, Raphael, 1968):

```
1: $OPEN \leftarrow \{s\}$ (prioritní fronta podle f), $CLOSED \leftarrow \emptyset$
2: $g(s) \leftarrow 0$, $f(s) \leftarrow h(s)$, ostatní $g \leftarrow \infty$
3: while $OPEN \neq \emptyset$ do
4: $n \leftarrow \arg \min_{x \in OPEN} f(x)$; odeber n z $OPEN$
5: if n je cíl then return rekonstruovaná cesta
6: end if
7: přidej n do $CLOSED$
8: for all $m \in succ(n)$ do
9: $g_{\text{new}} \leftarrow g(n) + w(n, m)$
10: if $g_{\text{new}} < g(m)$ then
11: $g(m) \leftarrow g_{\text{new}}$; $parent(m) \leftarrow n$; $f(m) \leftarrow g(m) + h(m)$
12: vlož/aktualizuj m v $OPEN$ (a odeber z $CLOSED$, byl-li tam)
13: end if
14: end for
15: end while
16: return „cesta neexistuje“
```

#### Přípustnost a konzistence heuristiky

**Přípustná** (*admissible*): nikdy nenadhodnocuje,  $h(n) \leq h^*(n)$  pro všechna  $n$ ;  $h^*(n)$  = skutečná cena nejlevnější cesty z  $n$  do cíle.

**Konzistentní** (*consistent*, monotónní): trojúhelníková nerovnost pro každou hranu  $(n, m)$

$$h(n) \leq w(n, m) + h(m), \quad h(g_{\text{cil}}) = 0.$$

**Vlastnosti:** konzistence  $\Rightarrow$  přípustnost. Přípustná  $h \Rightarrow A^*$  *optimální* (najde nejlevnější cestu). Konzistentní  $h \Rightarrow$  navíc  $f$  podél každé cesty neklesající a každý vrchol expandován *nejvýše jednou*.

**Speciální případy a varianty:**

- $h \equiv 0$  — A\* degeneruje na **Dijkstrův algoritmus**;
- heuristiky na mřížce: manhattanská  $|\Delta x| + |\Delta y|$  (4 směry), oktilová (8 směrů), eukleidovská;
- **Weighted A\***:  $f = g + \varepsilon h$ ,  $\varepsilon > 1$  — rychlejší, cesta nejvýše  $\varepsilon \times$  dražší;
- **IDA\***: šetří paměť ( $O(d)$  místo exponenciální).

### 3.3 Dynamické prostředí: D\* Lite

Agent za pohybu zjišťuje změny mapy (nová překážka, změněná cena). A\* od nuly v každém kroku = drahé  $\Rightarrow$  **inkrementální** algoritmy: při lokální změně přepočítají jen zasaženou část.

## LPA\* a D\* Lite

**LPA\*** (*Lifelong Planning A\**, Koenig, Likhachev, 2001) — inkrementální A\* pro pevný start a cíl. Ke každému vrcholu kromě  $g(n)$  i *lookahead* hodnota

$$rhs(n) = \begin{cases} 0 & n = s, \\ \min_{p \in pred(n)} (g(p) + w(p, n)) & \text{jinak.} \end{cases}$$

Vrchol **lokálně konzistentní**  $\Leftrightarrow g(n) = rhs(n)$ . Změna ceny hrany  $\Rightarrow$  některé vrcholy nekonzistentní  $\Rightarrow$  do prioritní fronty; změna se propaguje jen tam, kde je potřeba.

**D\* Lite** (Koenig, Likhachev, 2002) — LPA\* prohledávaný *od cíle ke startu* (hodnoty zůstanou použitelné, když se robot posune) + korekce prioritní fronty při pohybu. Funkčně ekvivalentní staršímu **D\*** (Stentz, 1994), ale výrazně jednodušší; *de facto* standard v mobilní robotice.

Neznámé prostředí: *optimistický předpoklad* (*freespace assumption*) — neznámé buňky = průchodné, agent jde po plánu, po nárazu přeplánuje; přesně scénář, pro nějž byl D\* navržen.

## 3.4 Hledání cest pro více agentů (MAPF)

♦ nad rámec slidů: není ve slidech, ale je to jediná vysloveně multiagentní část okruhu

### Multi-Agent Path Finding, MAPF

Vstup: graf  $G = (V, E)$  a  $k$  agentů, agent  $i$  má start  $s_i$  a cíl  $g_i$ . Čas diskrétní; v každém kroku každý agent buď přejde po hraně, nebo počká na místě.

**Konflikty** (řešení musí být *bezkonfliktní*): *vrcholový konflikt* — dva agenti ve stejném čase ve stejném vrcholu; *hranový* / „swap“ *konflikt* — dva agenti si v jednom kroku vymění pozice.

**Účelová funkce**: *sum-of-costs*  $\sum_i cost_i$ , nebo *makespan*  $\max_i cost_i$ . Nalezení *optimálního* řešení: NP-těžké.

**Prioritizované plánování** (rozvázaný přístup), varianta *Cooperative A\** (Silver, 2005):

- agenti uspořádání podle priority; pro  $i = 1, \dots, k$  cesta agenta  $i$  pomocí A\* v *prostoru-čase* (stav = dvojice ⟨vrchol, čas⟩), s vyhnutím již naplánovaným trajektoriím v **rezervační tabulce**;
- rychlé, škáluje, ale **neúplné** — existují řešitelné instance, které neřeší žádné pořadí priorit (dva agenti v úzkém koridoru, jeden musí uhnout do výklenku).

### Conflict-Based Search (CBS)

Sharon a kol., 2012–2015. Dvouúrovňový, *optimální* a *úplný*; vyhýbá se exponenciálnímu sdruženému stavovému prostoru  $V^k$ .

**Nízká úroveň**: jeden agent, optimální cesta v *prostoru-čase* pomocí A\* při respektování množiny *omezení* tvaru  $\langle a_i, v, t \rangle$  („agent  $i$  nesmí být ve vrcholu  $v$  v čase  $t$ “).

**Vysoká úroveň**: *strom omezení* prohledávaný podle ceny. Kořen = prázdná množina omezení + individuálně optimální cesty. Vezmi uzel s nejnižší cenou: cesty bezkonfliktní  $\Rightarrow$  řešení optimální; jinak první konflikt  $\Rightarrow$  *rozvětvení* na dva potomky, do každého omezení pro jednoho z kolidujících agentů a přeplánuje se *jen on*.

## 3.5 Shrnutí ke zkoušce

- A\*:  $f = g + h$ ; *přípustnost*  $h \leq h^* \Rightarrow$  optimalita; *konzistence*  $h(n) \leq w(n, m) + h(m) \Rightarrow$  každý vrchol se expanduje jednou.  $h \equiv 0$  dá Dijkstru; weighted A\* je  $\varepsilon$ -suboptimální.
- Dynamické prostředí: LPA\* (hodnoty  $g$  a  $rhs$ , lokální konzistence  $g = rhs$ ), D\* Lite (prohledávání od cíle, inkrementální oprava), *freespace assumption*.
- MAPF: vrcholové a hranové (swap) konflikty, *sum-of-costs* vs. *makespan*; optimální MAPF je NP-těžký.



- Prioritizované plánování / Cooperative A\* s rezervační tabulkou — rychlé, ale neúplné. CBS — nízká úroveň A\* s omezeními, vysoká úroveň větvení stromu omezení podle konfliktu; optimální a úplný.

## 4 Komunikace a znalosti v MAS: ontologie, řečové akty, FIPA-ACL, protokoly

### 4.1 Od agenta k multiagentnímu systému

Jeden agent umíme; systém agentů vyžaduje komunikaci, a tedy:

- **reprezentaci znalostí** — v čem jsou znalosti vyjádřeny;
- **společný slovník** — stejné slovo = stejný význam (*ontologie*);
- **standardní zprávy** — syntaxe a sémantika komunikace (*ACL*);
- **předvídatelné chování při komunikaci** — *interakční protokoly*;
- **technické problémy** — distribuovanost, mobilita agentů, asynchronnost, nespolehlivé kanály.

### 4.2 Co je ontologie

#### Ontologie

*Filozoficky*: nauka o bytí, Aristotelova „první filozofie“.

*V informatice*: **explicitní a formalizovaný popis části reality** — formální deklarativní popis s *glosářem* (definice konceptů) a *tezauzem* (definice vztahů mezi koncepty); slovník pro standardní ukládání a výměnu znalostí o doméně.

**Gruber (1995)**: „Ontologie je popis (podobný formální specifikaci programu) konceptů a vztahů, které mohou pro agenta nebo komunitu agentů formálně existovat.“ Zkráceně: *ontologie je explicitní specifikace konceptualizace*.

**Motivační příklad**: věta „7777 je nové CD kapely SRPR“ vyžaduje sdílenou konceptualizaci: 7777 = album (žánry, skladby, interpret); SRPR = kapela (členové, nástroje. . .).

#### Formalismus ontologie

- **Třídy** (*classes*, koncepty) — věci s něčím společným.
- **Instance** (*individuals*, objekty) — konkrétní jedinci tříd.
- **Vlastnosti** (*properties*) — atributy tříd a instancí.
- **Relace mezi třídami**, zejména **podtřída** (*is-a*) — *tranzitivní*.
- Další relace a vlastnosti (základní znalosti o doméně).

**Strukturální část** (třídy, relace, axiomy) = vlastní *ontologie*; + **fakta o konkrétních jedincích** = **znalostní báze** (*knowledge base*).

**Ontologie ontologií podle síly formalizace** (od slabých k silným): slovník (řízený slovník termínů) → glosář (významy v přirozeném jazyce) → tezaurus (synonyma) → neformální hierarchie (Amazon, wiki) → formální *is-a* hierarchie (subsumpce tříd) → třídy s vlastnostmi → omezení hodnot („každý člověk má právě jednu matku“) → libovolná logická omezení (složitě odvozovací algoritmy).

**Ontologie ontologií podle rozsahu:**

- **Aplikační** — v praxi nejběžnější, těžko znovupoužitelné.
- **Doménové** — předmět výzkumu sémantického webu.
- **Horní** (*upper ontologies*) — popis „všeho“ (Thing, Living thing, . . .), základ doménových ontologií: BFO, GFO, UFO; WordNet a IDEAS nevhodné pro formální odvozování.



### 4.3 Ontologické jazyky

Formální deklarativní jazyky pro reprezentaci znalostí: fakta + odvozovací pravidla; základ nejčastěji logika prvního řádu nebo *deskripční logika*.

**Rámce** (*frames*) — historický předchůdce (Minsky), klasická UI a expertní systémy; obdoba objektového programování:

| Terminologie rámců         | Objektová terminologie      |
|----------------------------|-----------------------------|
| Frame (rámec)              | třída objektů               |
| Slot                       | vlastnost / atribut objektu |
| Trigger, accessor, mutator | metoda                      |

**XML** — pro ontologie nevznikl, ale používá se: vlastní značky (<product>, <title>, <artist>) = slovník. Chybí sémantika — popisuje jen strukturu dokumentu, ne význam.

#### RDF — *Resource Description Framework*

Standard reprezentace znalostí (nejen) pro web. **Jednoduchý**: málo expresivní, rychlé odvozování.

Vše jako **trojice** *subjekt – predikát – objekt*, *Predikt(Subjekt, Objekt)*; např. *MarriedTo(Karel, Jja)*, *FatherOf(Karel, Pa)*. Množina trojic = orientovaný ohodnocený graf (*RDF graph*); zdroje identifikovány IRI. Serializace: RDF/XML, Turtle, N-Triples, JSON-LD. Dotazování: SPARQL. Nadstavba **RDFS**: třídy, *subClassOf*, *domain*, *range*.

#### OWL — *Web Ontology Language*

Původ v sémantickém webu; verze 1 a 2. *Kolekce několika formalismů* pro popis ontologií; syntaxe XML, RDF, funkcionální notace, Manchester syntax nebo Turtle. Cíl: formální, a přesto prakticky použitelný jazyk.

Základ: **deskripční logika** — *rozhodnutelný fragment* logiky prvního řádu.

**Předpoklad otevřeného světa** (*open world assumption*, OWA): co nevíme, je *nedefinované*; oproti uzavřenému světu (databáze, Prolog), kde neznámé = *nepravdivé*.

#### Deskripční logika

Entity: **koncepty** (třídy), **role** (vlastnosti, predikáty), **individa** (objekty). **Axiom** = logický výraz o rolích a konceptech; oproti rámcům a OOP (třída popsána *úplně*) popisují axiomaty třídu jen *částečně* (mohou přicházet z více zdrojů).

Rodina systémů podle povolených konstruktorů. Základní **ALC**: negace, konjunkce, disjunkce konceptů, omezené existenční a univerzální kvantifikátory. Rozšíření: hierarchie rolí ( $\mathcal{H}$ ), tranzitivita ( $\mathcal{S}$ ), inverzní role ( $\mathcal{I}$ ), kardinalitní omezení ( $\mathcal{N}$ ,  $\mathcal{Q}$ ), nominály ( $\mathcal{O}$ ).

Znalostní báze: **T-Box** (terminologie — axiomaty o konceptech a rolích) + **A-Box** (asertivní část — fakta o individuích).

**Profilý OWL**. *OWL 1 Lite (SHIF)* — nejjednodušší, blízko RDF, omezení kvůli rychlému strojovému zpracování; *OWL 1 DL (SHOIN)* — odpovídá deskripční logice (např. disjunktnost tříd), úplný a rozhodnutelný; *OWL 1 Full* — největší vyjadřovací síla, mnoho problémů nerozhodnutelných. *OWL 2 (SROIQ)* + profily: *EL* (polynomiální odvozování; velké lékařské ontologie typu SNOMED), *QL* (dotazy nad znalostní bází, napojení na relační databáze), *RL* (axiomaty ve tvaru pravidel).

#### KIF — *Knowledge Interchange Format*

Logika prvního řádu, LISPovská prefixová syntaxe. Projekt DARPA *Knowledge Sharing Effort*:

KIF = **vnitřní jazyk** zpráv (obsah), KQML = *vnější* (obálka). Příklady:

(salary 015-46-3946 widgets 72000)

```
(forall (?F ?T) (=> (and (instance ?F Farmer) (instance ?T Tractor)) (likes ?F ?T)))
```

**DAML+OIL** — DARPA Agent Markup Language + Ontology Interchange Language, přímý předchůdce OWL, opuštěn 2006.

#### 4.4 Uvažování o znalostech: epistemická logika

♦ nad rámec slidů: slidy modální logiky výslovně vynechávají; okruh ale mluví o „znalostech v MAS“, takže minimum se hodí umět]

##### Epistemická logika a společná znalost

Znalost agenta = modální operátor  $K_i\varphi$  („agent  $i$  ví, že  $\varphi$ “), **kripkovská sémantika možných světů**:  $K_i\varphi$  platí ve světě  $w \iff \varphi$  platí ve všech světech, které  $i$  neumí od  $w$  odlišit. **Znalost**: systém **S5**, klíčový axiom **T**:  $K_i\varphi \Rightarrow \varphi$  (co vím, platí). **Přesvědčení** ( $B_i$ , srov. BDI): systém **KD45**, místo T slabší **D**:  $\neg B_i \perp$  — konzistentní, ale *může být mylné*. Znalost vs. přesvědčení = přesně axiom T.

Skupina  $G$ :  $E_G\varphi$  — každý v  $G$  ví  $\varphi$ ; **společná znalost**  $C_G\varphi \equiv E_G\varphi \wedge E_GE_G\varphi \wedge \dots$  (všichni vědí, že všichni vědí, ...).

**Koordinovaný útok** (problém dvou generálů): domluva současného útoku přes nespolehlivého posla. Žádný konečný počet potvrzení nevytvoří společnou znalost — po  $k$  zprávách nejvýše  $E_G^k\varphi$ , nikdy  $C_G\varphi$ . Poučení: *nespolehlivá komunikace nikdy nevytvoří společnou znalost*, tedy ani dokonalou koordinaci.

#### 4.5 Proč je komunikace agentů jiná

OOP: „komunikace“ = volání metody, volaný *musí* vyhovět. Agenti nemohou jiného agenta přinutit ani mu měnit vnitřní proměnné — **provádějí komunikační akt** s cílem

- vyměnit si informace, nebo
- *ovlivnit* jiné agenty, aby něco udělali.

Příjemce má vlastní cíle a s informací, žádostí či dotazem naloží po svém: **komunikace v MAS je akce, ne volání**.

#### 4.6 Teorie řečových aktů

##### Řečové akty (*speech acts*) — Austin, 1962

Části užívání jazyka mají charakter **akcí** — mění stav světa jako fyzické akce:

- „Prohlašuji vás mužem a ženou.“
- „Vyhlašuji válku.“

*Pragmatická* teorie jazyka: řeč jako prostředek k dosahování cílů. Typická performativní slovesa: *request, inform, promise*.

##### Tři vrstvy řečového aktu

- **Lokuční akt** (*locutionary act*) — *co bylo řečeno*; samotný výrok. „Udělej mi čaj.“
- **Ilokuční akt** (*illocutionary act*) — *co bylo míněno*; lokuce + performativní význam (dotaz, žádost, ...). „Požádal mě o čaj.“
- **Perlokuční akt** (*perlocutionary act*) — *co se skutečně stalo*; efekt aktu. „Přiměla mě, abych jí uvařil čaj.“

Pro MAS klíčová **ilokuce** — obsah zprávy (performativa).

**Searle** — podmínky „zdařilosti“ řečového aktu; pro *MLUVČÍ* požádá *POSLUCHAČE* o akci:

- **Standardní vstupně-výstupní podmínky:** posluchač slyší, nejde o scénu ve filmu...
- **Předpoklady** (aby mluvčí mohl akt zvolit): posluchač je schopen akci provést; mluvčí věří, že je schopen; není zřejmé, že by ji provedl i bez požádání.
- **Upřímnost** (*sincerity*): mluvčí opravdu chce, aby se akce provedla.

#### Searlovy kategorie řečových aktů

Starší dělení: *request, advice, statement, promise, ...*

Novější (standardní) pět tříd:

- **Asertiva** (*assertives*, reprezentativa) — informují (*inform*).
- **Direktiva** (*directives*) — žádají o akci (*request*).
- **Komisiva** (*commissives*) — mluvčí se zavazuje (*promise*).
- **Expresiva** (*expressives*) — mentální stav, emoce („děkuji“).
- **Deklarace** (*declarations*) — mění stav věcí (válka, sňatek).

Dvě složky každého aktu: **performativní sloveso** (*request, query, inform*) + **propoziční obsah** („okno je zavřené“).

#### Plánovací teorie řečových aktů (Cohen, Perrault, 1979)

„...modelování [řečových aktů] v plánovacím systému jako operátorů definovaných... pomocí přesvědčení a cílů mluvčích a posluchačů. S řečovými akty se tak zachází stejně jako s fyzickými akcemi.“

Sémantika ve stylu **STRIPS** (předpoklady – delete – add) s modálními operátory pro *beliefs, abilities, wants*.

#### Příklad: Request a Inform.

|                        | Request(S,H,A)                                                   | Inform(S,H, $\varphi$ )                                |
|------------------------|------------------------------------------------------------------|--------------------------------------------------------|
| Předpoklad             | $(S \text{ believe } (H \text{ cando } A)) \wedge$               | $(S \text{ believe } \varphi)$                         |
| <i>cando</i>           | $(S \text{ believe } (H \text{ believe } (H \text{ cando } A)))$ |                                                        |
| Předpoklad <i>want</i> | $(S \text{ believe } (S \text{ want requestInstance}))$          | $(S \text{ believe } (S \text{ want informInstance}))$ |
| Efekt                  | $(H \text{ believe } (S \text{ believe } (S \text{ want } A)))$  | $(H \text{ believe } (S \text{ believe } \varphi))$    |

Efekt *Inform* není „*H* uvěří  $\varphi$ “, jen „*H* uvěří, že *S* věří  $\varphi$ “ — autonomní agent není povinen věřit, co mu kdo řekne (rozdíl oproti volání metody).

## 4.7 KQML a FIPA-ACL

#### KQML — Knowledge Query and Manipulation Language

90. léta, projekt DARPA *Knowledge Sharing Effort* (KSE), spolu s KIF.

- **KQML** = *vnější* jazyk („obálka dopisu“); *ilokuční* část zprávy.
- **KIF** = *vnitřní* jazyk; propoziční obsah, reprezentace znalostí.

Zpráva = *performativ* + pojmenované parametry:

```
(ask-one
 :content (PRICE IBM ?price)
 :receiver stock-server
 :language PROLOG
 :ontology NYSE-TICKS)
```

**Parametry:** content, force, reply-with, in-reply-to, sender, receiver, language, ontology.

**Performativa:** achieve, advertise, ask-about, ask-one, ask-all, ask-if, break, sorry, error, broadcast, forward, recruit-one, recruit-all, reply, subscribe, tell, ...

**Kritika KQML:** chybí performativ *commit* (závazek); nepřesná sémantika; nesystematická a zbytečně velká množina performativ; nestandardizovaný transport.

## FIPA-ACL — Agent Communication Language

FIPA (*Foundation for Intelligent Physical Agents*, dnes standardizační komise IEEE): ACL = **zjednodušení a zpřesnění KQML** — menší, systematictější množina performativ, *formálně definovaná sémantika*. Implementace: platforma JADE.

**Struktura zprávy** (13 polí — performativ + 12 parametrů; povinné jen **performative**):

| Pole                                     | Význam                                                  |
|------------------------------------------|---------------------------------------------------------|
| performative                             | typ komunikačního aktu (ilokuční síla) — <i>povinné</i> |
| sender, receiver, reply-to               | identifikátory agentů (AID)                             |
| content                                  | vlastní obsah zprávy                                    |
| language                                 | jazyk obsahu (SL, KIF, ...)                             |
| encoding, ontology                       | kódování a ontologie pro interpretaci obsahu            |
| protocol                                 | interakční protokol, jehož je zpráva součástí           |
| conversation-id, reply-with, in-reply-to | párování zpráv do konverzace                            |
| reply-by                                 | termín odpovědi                                         |

(inform  
:sender agent1  
:receiver agent2  
:content (price good2 150)  
:language sl  
:ontology hpl-auction)

**Performativa FIPA-ACL** (22 celkem, nejdůležitější):

| Skupina             | Performativa                                                                 |
|---------------------|------------------------------------------------------------------------------|
| předávání informace | inform, inform-if, inform-ref, confirm, disconfirm                           |
| žádost o informaci  | query-if, query-ref, subscribe                                               |
| žádost o akci       | request, request-when, request-whenever, cancel                              |
| vyjednávání         | cfp ( <i>call for proposals</i> ), propose, accept-proposal, reject-proposal |
| provádění akcí      | agree, refuse, failure                                                       |
| chyby, směrování    | not-understood, proxy, propagate                                             |

♦ nad rámec slidů: formální sémantika SL ve slidech není, performativa ano]

### Sémantika FIPA-ACL: FP a RE

Jazyk SL (*Semantic Language*) — modální logika s operátory  $B_i\varphi$  ( $i$  věří),  $U_i\varphi$  (je nejistý),  $I_i\varphi$  (má záměr), Feasible, Done. Ke každému performativu:

- **FP** (*feasibility precondition*) — co musí platit, aby odesílatel mohl akt provést;
- **RE** (*rational effect*) — efekt, kvůli němuž akt provádí; příjemce *není povinen* RE naplnit (autonomie) — RE je jen důvod odesílatele.

Pro  $\langle i, \text{inform}(j, \varphi) \rangle$ :

$$\text{FP: } B_i\varphi \wedge \neg B_i(Bif_j\varphi \vee Uif_j\varphi), \quad \text{RE: } B_j\varphi.$$

Čteno: „věřím  $\varphi$  a nevěřím, že  $j$  už o  $\varphi$  ví ( $Bif$  — ví, zda platí) ani o tom má nejisté mínění ( $Uif$ ); chci, aby  $j$  uvěřil  $\varphi$ .“ Pro  $\langle i, \text{request}(j, \alpha) \rangle$ : FP:  $FP(\alpha)[i \setminus j] \wedge B_i \text{Agent}(j, \alpha) \wedge \neg B_i I_j \text{Done}(\alpha)$ , RE:  $\text{Done}(\alpha)$ .

**Praktický problém:** sémantika neverifikovatelná — zvenčí nelze ověřit, že agent věří tomu, co tvrdí (*sincerity assumption*); slabina mentalistické sémantiky ACL. Alternativa: *sociální sémantiky nad veřejnými závazky* (*commitments*).

## 4.8 Interakční protokoly

Zpráva sama nestačí — agenti vedou **konverzace** s předepsanou strukturou. Interakční protokol (*interaction protocol*) = vzor přípustných posloupností zpráv; FIPA standardizuje kolem tuctu (specifikace FIPA-IP, notace AUMML).

| Protokol                          | Průběh                                                                                                                                                                                                  |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FIPA-Request</b>               | $I \rightarrow P$ : request. $P \rightarrow I$ : refuse (konec), nebo agree; po vykonání inform-done / inform-result, případně failure. Možná odpověď i not-understood.                                 |
| <b>FIPA-Query</b>                 | $I \rightarrow P$ : query-if / query-ref. $P$ : refuse nebo agree, dále inform s výsledkem nebo failure.                                                                                                |
| <b>FIPA-Contract-Net</b>          | viz níže: $cfp \rightarrow n \times (\text{propose} / \text{refuse}) \rightarrow \text{accept-proposal}$ vítězi + $\text{reject-proposal}$ ostatním $\rightarrow \text{inform-done} / \text{failure}$ . |
| <b>FIPA-Iterated-Contract-Net</b> | jako CNET, iniciátor může po nabídkách vyhlásit <i>další kolo</i> cfp (vyjednávání o lepších podmínkách).                                                                                               |
| <b>FIPA-Subscribe</b>             | $I$ : subscribe; $P$ : inform při každé změně sledované hodnoty, dokud nepřijde cancel.                                                                                                                 |
| <b>FIPA-Brokering Recruiting</b>  | / $I$ požádá zprostředkovatele ( <i>broker</i> ); ten najde poskytovatele, deleguje požadavek a vrátí výsledky (příp. nechá odpovědět přímo).                                                           |
| <b>Aukční protokoly</b>           | FIPA-English-Auction, FIPA-Dutch-Auction (viz odst. 5.11).                                                                                                                                              |

**Poznámka k návrhu.** Protokol určuje legální zprávy v daném stavu konverzace — *předvídatelnost* autonomního chování; agent implementovatelný jako konečný automat nad stavy konverzace (JADE: třídy AchieveREInitiator, ContractNetResponder apod.).

## 4.9 Shrnutí ke zkoušce

- Ontologie = explicitní formalizovaný popis části reality (glosář + thesaurus); Gruberova definice. Formalismus: třídy, instance, vlastnosti, relace (tranzitivní *is-a*), fakta. Ontologie + fakta = znalostní báze.
- Škála formalizace od slovníku po libovolná logická omezení; aplikační / doménové / horní ontologie.
- RDF = trojice *subjekt-predikát-objekt*, jednoduché a rychlé; RDFS přidává třídy.
- OWL = deskripční logika (rozhodnutelný fragment FOL), *open world assumption*, T-Box vs. A-Box; profily Lite/DL/Full (*SHIF*, *SHOIN*), OWL 2 = *SRIOQ* + profily EL/QL/RL.
- KIF = FOL v LISPovské syntaxi, obsah zprávy; KQML = obálka.
- Epistemická logika:  $K_i\varphi$  + možné světy; znalost = S5 (axiom T), přesvědčení = KD45. Společná znalost  $C_G\varphi$  nevznikne nespolehlivou komunikací (koordinovaný útok).
- Komunikace agentů = *akce*, ne volání metody — příjemce není povinen vyhovět.
- Austin: řečové akty; lokuce / ilokuce / perlokuce. Searle: podmínky zdařilosti, pět kategorií (asertiva, direktiva, komisiva, expresiva, deklarace); performativní sloveso + propoziční obsah.
- Cohen & Perrault: sémantika řečových aktů ve stylu STRIPS (pre / efekt) s modálními operátory; umět Request(S,H,A) a Inform(S,H, $\varphi$ ) — efekt Inform =  $H$  věří, že  $S$  věří  $\varphi$ .
- KQML (obálka) + KIF (obsah); parametry a performativa; kritika (chybí commit, nejasná sémantika).
- FIPA-ACL: struktura zprávy (performative, sender/receiver, content, language, ontology, protocol, conversation-id, reply-by), 22 performativ, sémantika FP a RE v jazyce SL; neverifikovatelnost.
- Protokoly: FIPA-Request, FIPA-Query, Contract Net (+ iterovaný), Subscribe, Brokering, aukční protokoly.

## 5 Distribuované řešení problémů, kooperace, teorie her a aukce

Nejobsáhlejší věta okruhu, dvě situace  $\Rightarrow$  dvě poloviny kapitoly:

- odst. 5.1–5.6: agenti *sdílejí cíl*, jen dělba práce — *distribuované řešení problémů, kooperace*;
- odst. 5.7–5.13: agenti s *vlastními zájmy* — teorie her (Nashova ekvilibria, Paretova efektivita), mechanismy odolné vůči manipulaci (alokace zdrojů, aukce).

## 5.1 Koherence a koordinace

Agenti: různé cíle, autonomní, pracují v čase, rozhodují za běhu  $\Rightarrow$  nutná *dynamická koordinace*.  
Sdílejí **úlohy** (*task sharing*) a **informace** (*result sharing*).

### Koherence a koordinace

- **Koherence** (*coherence*) — kvalita systému *jako celku*: kvalita řešení, využití zdrojů, degradace při poruchách.
- **Koordinace** (*coordination*) — *minimalizace režie* synchronizace: bez duplicitní práce, konfliktů o zdroje, zbytečné komunikace.
- Koherence možná i při špatné koordinaci (práce třikrát, výsledek správný) — plýtvání.

### CDPS — Cooperative Distributed Problem Solving

Lesser a kol., 80. léta. Kooperace na problému přesahujícím individuální schopnosti (informace, zdroje, výpočetní kapacita).

- Předpoklad: agenti *implicitně sdílejí společný cíl*, bez konfliktů — **benevolence**.
- Měřítka úspěchu: *výkon celku*; agent pomáhá celku i ve svůj neprospěch.
- Benevolence *enormně zjednodušuje* návrh.

|                                                   | Charakteristika                                                             | Rozdíl                                                                                      |
|---------------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>PPS</b><br>( <i>parallel problem solving</i> ) | Bond, Gasser, 80. léta; paralelní řešení, homogenní jednoduché procesory    | agenti nejsou autonomní — v podstatě paralelní algoritmus                                   |
| <b>CDPS</b>                                       | heterogenní agenti se sofistikovanými schopnostmi, sdílený cíl, benevolence | konflikty se neřeší, protože nevznikají                                                     |
| <b>MAS</b>                                        | agenti s <i>vlastními, nesdílenými</i> cíli                                 | nutno řešit: proč a jak kooperovat, identifikace a řešení konfliktů, vyjednávání, smlouvání |

## 5.2 Sdílení úloh a sdílení výsledků

Obecný postup CDPS — tři fáze:

1. **Dekompozice problému** — hierarchická, rekurzivní; *jak* rozložit, *kdo* rozkládá. Krajní varianta model ACTORS: nový agent na každý podproblém, až po jednotlivé instrukce.
2. **Řešení podproblémů** — sdílení informací, případně synchronizace akcí.
3. **Syntéza řešení** — hierarchické skládání dílčích výsledků.

**Sdílení úloh**: nutná *dohoda* (kdo co udělá), typicky Contract Net. **Sdílení výsledků**: *proaktivní* (pošlu, co se podle mě bude hodit), nebo *reaktivní* (na vyžádání).

### Přínosy sdílení výsledků

- **Jistota** (*confidence*) — porovnání nezávislých řešení; shoda zvyšuje důvěru.
- **Úplnost** (*completeness*) — *lokální* pohledy  $\Rightarrow$  globálnější představa o problému.
- **Přesnost** (*precision*) — zpřesnění celkového řešení.
- **Včasnost** (*timeliness*) — kratší čas díky distribuci.

## 5.3 Contract Net

### Contract Net Protocol (CNET)

Smith a Davis, 1977–1980. **Sdílení úloh pomocí smluv** podle vzoru trhu: agent v nouzi = *manažer* (*manager*), ostatní = *dodavatelé* (*contractors*). Čtyři fáze:

1. **Rozpoznání** (*recognition*) — problém nad vlastní síly (chybí schopnost, zdroje, informace,

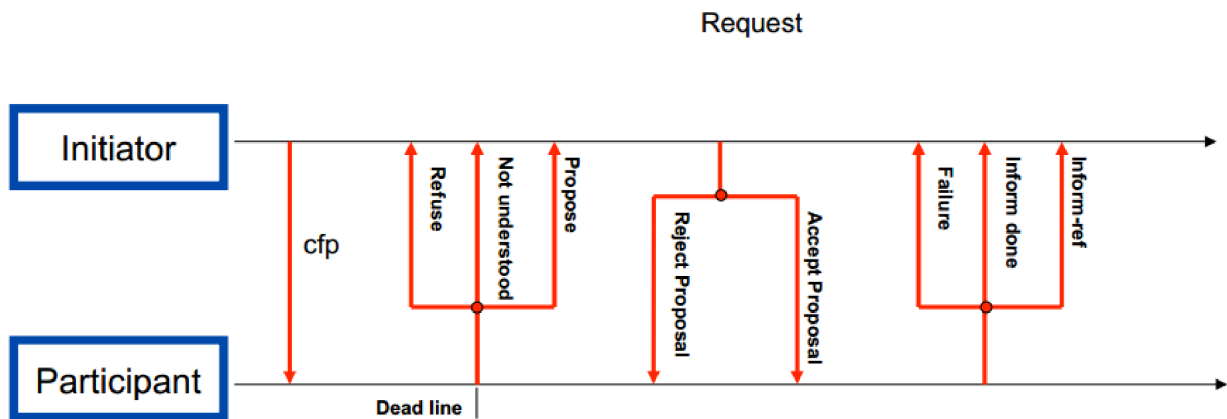


nebo by řešení bylo horší).

2. **Ohlášení** (*announcement*) — typicky broadcast: *popis úlohy* (i spustitelný), *omezení* (termín, cena, kvalita), *meta-informace* (kdo se smí hlásit).
3. **Nabídky** (*bidding*) — zájemci se schopnostmi pošlou nabídku (*tender*): cena / čas / kvalita.
4. **Přidělení a realizace** (*awarding, expediting*) — manažer vybere vítěze, smlouva; vítěz vykoná, vrátí výsledek.

**Vlastnosti:** jednoduchý, decentralizovaný, robustní vůči výpadkům; dodavatel může být manažerem podúlohy  $\Rightarrow$  *hierarchické kaskády subkontraktů*; dynamické rozdělení zátěže. Nejčastěji implementovaný koordinační mechanismus vůbec (FIPA: FIPA-Contract-Net; JADE: hotové chování).

**Limity:** drahý broadcast při mnoha agentech; předpoklad benevolence (dodavatel nelže o schopnostech) — jinak aukce odolné vůči manipulaci (odst. 5.11).



*Contract Net jako interakční protokol FIPA-ACL. Iniciátor rozešle cfp (call for proposals) s termínem; účastník odpoví not-understood, refuse, nebo propose. Iniciátor pak reject-proposal, nebo accept-proposal; vítěz hlásí failure, inform-done, nebo inform-ref s výsledkem.*

## 5.4 Systémy s tabulí (blackboard)

### Blackboard system (BBS)

Vůbec první schéma kooperativního řešení problémů (HEARSAY-II, rozpoznávání řeči, 70. léta). Sdílení výsledků přes společnou datovou strukturu — **tabuli** (*blackboard*).

Metafora: experti kolem tabule čtou i píší; úlohy se objevují dynamicky; kdo úlohu umí, zapíše částečné řešení; opakuje se, dokud není na tabuli řešení celé úlohy.

**Role:**

- **Tabule** — sdílená paměť, typicky několik *úrovní abstrakce* (hierarchie cílů a akcí); zásadní formalismus reprezentace informace.
- **Experti** (*knowledge sources*) — řeší konkrétní typ úlohy; reagují na cíle na tabuli, cíl z ní přebírají, po vybrání vykonají akci; seznam schopností a efektivita.
- **Arbitr** (*arbiter, control component*) — vybírá, kdo smí k tabuli; čistě reaktivní, nebo uvažuje plány a maximalizuje očekávanou utilitu; odpovídá za řešení na vyšší úrovni.

**Nutnost vzájemného vyloučení nad tabulí** — úzké hrdlo systému.

**Klady a zápory BBS:**

- + jednoduchá koordinace, kooperace, sdílení úloh i výsledků;
- + experti o sobě *nemusí vědět* (*loose coupling*); nový expert bez změny ostatních;
- + zprávy na tabuli lze přepisovat — delegace úloh, podúlohy, změna expertů;
- všichni potřebují přístup k tabuli a stejnou architekturu; kolem tabule „narováno“ (částečně řeší distribuované hašovací tabulky).



## Příklady.

- *BBWar*: tabule mapuje požadované schopnosti na úlohy. *Velitel*: „dobyj město“ (ATTACK-CITY) → několik „zaútoč na pozici“ (ATTACK-LOCATION). Vojáci: výběr mise podle schopností.
- *FELINE* (Wooldridge, Jennings, 1990): distribuovaný expertní systém před ACL; agent zveřejňuje *Skills* (co umí dokázat) a *Interests* (co ho zajímá); komunikace trojicemi (odesílatel, příjemce, hypotéza + řečový akt).

## 5.5 Distribuované omezující podmínky (DisCSP a DCOP)

♦ nad rámec slidů: není ve slidech; okruh ale mluví o „distribuovaném řešení problémů“, kam formální větve patří

### DisCSP a DCOP

- **Distribuovaný CSP**: proměnné  $x_1, \dots, x_n$  s doménami  $D_i$  rozděleny mezi agenty (typicky jedna na agenta); omezení mezi proměnnými *různých* agentů. Cíl: přiřazení splňující všechna omezení. Nikdo nevidí celý problém; komunikace jen se sousedy v grafu omezení.
- **DCOP** — optimalizační varianta: *měkká* omezení jako funkce ceny  $f_{ij}(x_i, x_j)$ , maximalizuje se  $\sum_{ij} f_{ij}$  = sociální blahobyt.
- **Algoritmy**: *ABT* (asynchronní backtracking, Yokoo) — uspořádaní agenti, zprávy ok? (přiřazení níž) a *nogood* (nahoru při konfliktu); dále *ADOPT*, *DPOP*, *Max-Sum*.
- Aplikace: rozvrhování schůzek, přidělování úkolů senzorům, alokace frekvencí.

## 5.6 Interakce agentů v prostředí

Pomoc i překážení i *bez komunikace*, jen přes prostředí:

- pozitivní interakce — roboti přenesou cihlu, jen když tlačí ze dvou stran (nutná synchronizace);
- negativní interakce — roboti natlačení ke dveřím je nemohou otevřít (nutná koordinace).

**Bez znalosti typů interakcí** v daném MAS nelze navrhnout efektivní řídicí mechanismy. Nejjednodušší model: dva racionální (sobečtí) agenti v prostředí připomínajícím hru.

## 5.7 Sobecký agent

Proč být upřímný o schopnostech? Proč dokončit přidělenou úlohu? Homogenní systém (celý náš návrh) ⇒ benevolence dobrá strategie. Většinou ale různé zájmy — konflikt společného cíle a cílů jednotlivců (řízení letového provozu: všichni chtějí bezpečnost, každá aerolinka chce přistát první); někdy společný zájem ani není zřejmý ⇒ **sobecké** (*self-interested*) agenty.

### Sobecký agent

Množina výsledků  $O = \{o_1, o_2, \dots\}$  *společná*, preference nikoli: agent  $i$  má vlastní **utilitní funkci**  $u_i : O \rightarrow \mathbb{R}$ , která výsledky uspořádá.

**Strategické myšlení**: maximalizuj svou utilitu za předpokladu racionality všech ostatních. *Nemaximalizuje* společnou utilitu, ale je *robustní*.

## 5.8 Hra v normální formě

### Hra dvou hráčů v normální formě

Agenti  $i, j$  s množinami akcí  $A_i, A_j$ ; výsledek určen oběma:  $e : A_i \times A_j \rightarrow O$ . **Strategie**  $s_i$ :

- **ryzí** (*pure*) — deterministická volba jedné akce;
- **smíšená** (*mixed*) — rozdělení pravděpodobnosti nad ryzími (náhodný výběr).

**Výplatní matice** (*payoff matrix*): řádky = strategie  $i$ , sloupce = strategie  $j$ , buňka = dvojice  $u_i/u_j$ . Obecně: trojice  $\langle N, (A_i)_{i \in N}, (u_i)_{i \in N} \rangle$ ,  $N$  = množina hráčů.

## Dominance

$s_i$  **silně dominuje**  $s'_i$ : striktně lepší výsledek proti všem strategiím protihráče. **Slabě dominuje**: alespoň stejně dobrý proti všem, striktně lepší alespoň proti jedné (definice přednášky).

**Dominantní** strategie: dominuje všem ostatním strategiím  $i$ .

- Podstatný rozdíl: pravdomlupnost ve Vickreyově aukci je jen *slabě* dominantní (odst. 5.11).
- Racionální agent hraje dominantní strategii, existuje-li.
- *Iterované odstraňování dominovaných strategií* (IESDS) — základní řešící technika: dominovanou strategii nikdo nezahraje  $\Rightarrow$  vyškrtnout z matice  $\Rightarrow$  další strategie se mohou stát dominovanými. *Silně* dominované: pořadí nerozhoduje, výsledek jednoznačný. *Slabě* dominované: na pořadí **záleží**, lze přijít o ekvilibria.

## Nashovo ekvilibrium

$(s_1^*, s_2^*)$  je **Nashovo ekvilibrium** (NE), pokud

- hraje-li  $i$  strategii  $s_1^*$ , je pro  $j$  nejlepší  $s_2^*$ , a zároveň
- hraje-li  $j$  strategii  $s_2^*$ , je pro  $i$  nejlepší  $s_1^*$

— *vzájemně nejlepší odpovědi* (*mutual best responses*). Pro  $n$  hráčů: profil  $s^* = (s_1^*, \dots, s_n^*)$  je NE, pokud pro každého  $i$  a každou jeho  $s_i$

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*),$$

$s_{-i}$  = strategie ostatních. **Žádný hráč nemá jednostranný důvod se odchýlit.**

- Naivní hledání:  $m^n$  profilů ( $n$  agentů,  $m$  strategií).
- Definice pro *ryzí* strategie: NE v ryzích nemusí existovat (kámen–nůžky–papír), nebo jich je více.

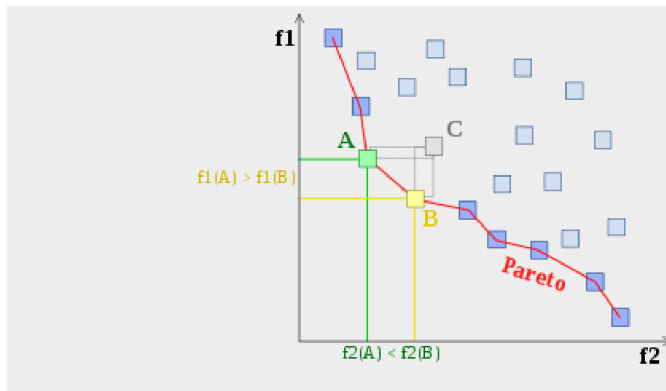
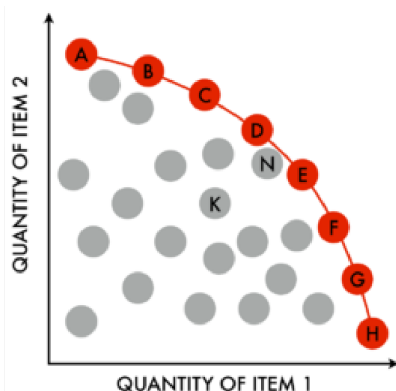
**Věta 5.1** (Nashova věta, 1950). *Každá hra s konečným počtem hráčů a strategií má alespoň jedno NE ve smíšených strategiích.*

**Výpočetní složitost**: hledání NE je *totální* vyhledávací problém (řešení vždy existuje), **PPAD-úplný** (Daskalakis, Goldberg, Papadimitriou pro  $\geq 3$  hráče; Chen a Deng pro dva) — „o něco méně beznadějně než NP-úplnost“.

## Paretova efektivita a sociální blahobyť

**Pareto-optimální** profil (*Pareto efficient*): neexistuje jiný profil zlepšující výsledek jednoho agenta bez zhoršení jiného. Ne-Pareto-optimální řešení lze zlepšit „zadarmo“.

**Sociální blahobyť** (*social welfare*):  $sw(o) = \sum_{i \in N} u_i(o)$ . Maximalizace má smysl v kooperativních scénářích (jeden tým, jeden vlastník, jedna úloha; čím homogennější systém, tím lépe). Maximum  $sw$  je vždy Pareto-optimální, opačně ne.



**Paretova mez.** Vlevo: červené body A–H Pareto-optimální — víc jedné položky jen za cenu druhé; šedé body uvnitř (např. K) nikoli — existuje bod lepší v obou souřadnicích. Vpravo: totéž pro kritériální funkce  $f_1, f_2$  — A dominuje C, mez = množina nedominovaných řešení.

## 5.9 Klasické hry

### Věžňovo dilema (*Prisoner's dilemma*)

| $i \backslash j$ | $C$ (spolupráce) | $D$ (zrada) |
|------------------|------------------|-------------|
| $C$              | 3/3              | 0/4         |
| $D$              | 4/0              | 1/1         |

$C = cooperate$  (držet se spolupachatelem),  $D = defect$  (udat ho, nižší trest). Kanonické výplaty:  $R$  (reward, oboustranná spolupráce),  $P$  (punishment, oboustranná zrada),  $T$  (temptation, zradím spolupracujícího),  $S$  (sucker's payoff, jsem zrazen). Podmínky:

$$T > R > P > S \quad \text{a často navíc} \quad 2R > T + S.$$

- $R > P$ : oboustranná spolupráce lepší než oboustranná zrada.
- $T > R, P > S$ : zrada je dominantní pro oba.
- $(D, D)$ : **jediné NE** a zároveň **jediné ne-Pareto-optimální** řešení.
- $(C, C)$ : maximum sociálního blahobytu ( $3 + 3 = 6$ ).

**Individuální racionalita  $\Rightarrow$  kolektivně nejhorší výsledek.**

### Důsledky věžňova dilematu:

- **Tragédie obecní pastviny** (*tragedy of the commons*): sdílený zdroj (pastvina, rybolov, atmosféra) vyčerpán — každý maximalizuje svůj užitek.
- Co znamená být racionální; jsou lidé racionální? (Experimentálně spolupracují častěji, než teorie předpovídá.)
- **Stín budoucnosti** (*shadow of the future*): v iterovaném PD se situace mění.

### Iterované věžňovo dilema a Axelrodovy turnaje

- Konečně mnoho kol a oba to vědí: zpětná indukce  $\Rightarrow D$  v každém kole (v posledním není důvod spolupracovat, tedy ani v předposledním, ...).
- Nekonečně / neznámý konec (pravděpodobnost pokračování): spolupráce racionální — *folk theorem*: každý výplatní profil lepší než minimax lze podepřít jako ekvilibrium opakované hry.

Axelrodovy turnaje pro iterované PD (1980): dvakrát vyhrála nejjednodušší přihlášená strategie **Tit For Tat** (TFT, „půjčka za oplátku“) — *první kolo spolupracuj, pak opakuj poslední tah soupeře*. Vlastnosti úspěšných strategií:

- **slušnost** (*nice*) — nezrazuj první;
- **odvetnost** (*retaliating*) — na zradu ihned odpověz;
- **odpouštění** (*forgiving*) — po návratu ke spolupráci odpusť;
- **nezávistivost** (*non-envious*) — neporážej *konkrétního* soupeře; TFT nikdy nezískal víc bodů než protihráč, přesto vyhrál;
- pátá: **srozumitelnost** (*clarity*) — čitelná, jinak nelze navázat spolupráci.

Slabina TFT: šum (chybný tah)  $\Rightarrow$  dva TFT uvíznou ve vzájemné odvetě  $\Rightarrow$  varianty *Generous TFT* (občas odpustí), *Tit for Two Tats*.

### Koordinální hra

| $i \backslash j$ | Balet | Zápas |
|------------------|-------|-------|
| Balet            | 1/2   | 0/0   |
| Zápas            | 0/0   | 2/1   |

Též *Battle of the Sexes* (BoS), *Bach or Stravinsky*. Výplaty podporují *shodu* strategií; hráči se liší v tom, na které shodě jim záleží víc.

- Ryzí NE: (Balet, Balet), (Zápas, Zápas) — zároveň maxima sw i Pareto-optima.
- **Smíšené NE**: každý hraje *preferovanější* možnost s  $2/3$ , druhou s  $1/3$ ; očekávaná utilita jen

$2/3$  — méně než v kterémkoli ryzím NE.

Výpočet smíšeného NE (recept pro  $2 \times 2$ ):  $j$  volí pravděpodobnost  $q$  Baletu tak, aby byl  $i$  indiferentní:

$$u_i(\text{Balet}) = 1 \cdot q + 0 \cdot (1 - q), \quad u_i(\text{Zápas}) = 0 \cdot q + 2 \cdot (1 - q).$$

$q = 2 - 2q \Rightarrow q = 2/3$ :  $j$  hraje Balet (svou preferenci) s  $2/3$ ; symetricky  $i$  hraje Zápas s  $2/3$ . Očekávaná utilita  $i$ :  $1 \cdot \frac{2}{3} = \frac{2}{3}$ . **Klíčové pravidlo: pravděpodobnosti volím tak, aby byl protihráč indiferentní.**

**Antikoordinační hra** (*Chicken, Hawk–Dove*): výplaty podporují různé strategie:  $u(\text{uhnu}, \text{uhnu}) = 5/5$ ,  $u(\text{uhnu}, \text{nedám se}) = 1/6$ ,  $u(\text{nedám se}, \text{uhnu}) = 6/1$ ,  $u(\text{nedám se}, \text{nedám se}) = 0/0$ . Ryzí NE: obě *asymetrické* dvojice; maximum sw (oba uhnou,  $5+5$ ) *není* NE. Symetrické smíšené NE:  $p = 1/2$ , očekávaná utilita 3.

## 5.10 Sociální volba a volby

◆ nad rámec slidů: ve slidech je celá kapitola, oficiální znění okruhu ji ale nejmenuje — zařazeno jako doplněk k agregaci preferencí

Teorie her: chování agentů při daných preferencích. *Teorie sociální volby* opačně: z individuálních preferencí *odvodit* kolektivní rozhodnutí. **Funkce sociálního blahobytu** agreguje preference do úplného uspořádání  $>_{sw}$ ; **funkce sociální volby** vybírá jednoho vítěze.

**Volební systémy:**

- *většinový* — nad 50 %;
- *prostá většina (plurality)* — bod za každé první místo;
- *prostá většina s vyřazováním* (IRV) — opakovaně vyřadit nejslabšího, přerozděl jeho hlasy;
- *sekvenční většinové volby* — pavouk párových soubojů; *pořadí zápasů ovlivňuje výsledek*;
- *Bordova metoda* —  $N - 1$  bodů za první místo,  $\dots$ , 0 za poslední; konsenzuální, nikoli většinové hlasování;
- *Slaterův systém* — acyklické uspořádání minimalizující počet obrácených hran v turnajovém grafu; NP-těžké.

Vítěz prosté většiny nemusí být většinový:  $o_1 = 40\%$ ,  $o_2 = 30\%$ ,  $o_3 = 30\% \Rightarrow$  vyhrává  $o_1$ , ač ho 60 % voličů nechtělo  $\Rightarrow$  **taktické hlasování**.

### Condorcetův paradox

Ať zvolíme kterýkoli výsledek, většina voličů je nespokojena. *Příklad*:  $V_1 : A > B > C$ ,  $V_2 : B > C > A$ ,  $V_3 : C > A > B$ . Párové souboje:  $A$  porazí  $B$  (2:1),  $B$  porazí  $C$  (2:1),  $C$  porazí  $A$  (2:1)  $\Rightarrow$  **kolektivní preference cyklická**, ač individuální jsou lineární; *Condorcetův vítěz* neexistuje. Stejný motiv jako neexistence ryzího NE v kámen–nůžky–papír.

### Kritéria volebních systémů

- **Paretova podmínka** (jednomyslnost):  $X$  před  $Y$  u *všech* voličů  $\Rightarrow X >_{sw} Y$ . *Splňuje* majority a Borda, *nesplňuje* sekvenční většinové.
- **Podmínka Condorcetova vítěze**: kdo porazí všechny v párovém srovnání, vyhrává. *Splňuje* sekvenční většinové, *nesplňuje* plurality ani Borda.
- **Nezávislost na irelevantních alternativách** (IIA):  $X >_{sw} Y$  závisí jen na relativním pořadí  $X$  a  $Y$  u voličů. *Nesplňuje* prakticky žádný běžný systém.
- **Neomezený definiční obor** (univerzalita), **nediktátorství**.

**Věta 5.2** (Arrowova věta o nemožnosti, 1951). *Pro tři a více kandidátů neexistuje preferenční volební systém, který by převedl individuální uspořádání na úplné a tranzitivní společenské uspořádání a zároveň splňoval neomezený definiční obor, nediktátorství, Paretovu podmínku a IIA.*

Jednodušeji: pro  $> 2$  kandidáty je jediná procedura s Paretovou podmínkou a IIA *diktatura*. *Negativní* výsledek o limitech kolektivního rozhodování: neplyne, že jediný funkční systém je diktatura, ale že každý reálný systém některé kritérium obětuje (nejčastěji IIA).

**Gibbardova–Satterthwaiteova věta:** každá surjektivní nediktátorská funkce sociální volby pro  $\geq 3$  kandidáty je **manipulovatelná**. Pro MAS zásadní: agent = počítač, taktické hlasování mu nedělá morální problém  $\Rightarrow$  hledají se mechanismy odolné vůči manipulaci — téma aukcí.

## 5.11 Aukce jako mechanismus alokace zdrojů

### Aukce

**Tržní instituce:** zprávy obchodníků nesou cenovou informaci — nabídka ke koupi (*bid*) nebo k prodeji (*ask*) za danou cenu; přednost mají vyšší nabídky ke koupi a nižší k prodeji.

- V MAS především **mechanismus rozdělení vzácných zdrojů mezi agenty**.
- *Reálně:* eBay, Sotheby's, těžební povolení, frekvenční pásma pro operátory. *V informatice:* procesorový čas, šířka pásma, výpočetní úlohy, reklamní pozice.
- **Efektivita aukce:** zdroj získá agent s nejvyšším oceněním. Prodávající *maximalizuje* cenu, kupující *minimalizuje*; každý kupující má vlastní utilitu.

### Klasifikace aukcí

**Podle ocenění předmětu:**

- *soukromá hodnota* (*private value*) — závisí jen na kupujícím (lístek na koncert);
- *společná hodnota* (*common value*) — pro všechny stejná, ale neznámá (těžební práva)  $\Rightarrow$  *prokletí vítěze* (*winner's curse*): vyhrává ten, kdo se nejvíc spletl směrem nahoru;
- *korelovaná hodnota* — kombinace.

**Podle protokolu:** vítěz platí první / druhou cenu; otevřené vyvolávání (*open cry*) vs. obálková metoda (*sealed bid*); jedno vs. více kol.

## 5.12 Čtyři základní typy aukcí

| Aukce                                                   | Protokol                                                                     | Racionální strategie                                                                                                | Poznámky                                                                                                                      |
|---------------------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Anglická</b>                                         | otevřené vyvolávání, rostoucí cena, platí se první (nejvyšší) cena           | <i>dominantní:</i> přiřazovat po $\varepsilon$ až do svého ocenění, pak odejít                                      | Nejběžnější (starožitnosti, umění, internet — asi 95 % aukcí). Kupující typicky přepatí.                                      |
| <b>Holandská</b>                                        | otevřené vyvolávání, klesající cena, první, kdo přijme, platí aktuální cenu  | <i>dominantní neexistuje;</i> čekat těsně pod své ocenění — hazard                                                  | Rychlá; zboží podléhající zkáze (květiny, ryby). Oblíbená u prodávajících. Neodhalí tržní cenu ani preference ostatních.      |
| <b>FPSB</b><br>( <i>first-price sealed bid</i> )        | jedno kolo, uzavřené obálky, nejvyšší nabídka vyhrává a platí svou cenu      | <i>dominantní neexistuje;</i> nabídnout méně než ocenění ( <i>bid shading</i> ), optimum závisí na odhadu ostatních | Nemovitosti, státní dluhopisy. Nenutí použít utilitní funkci, neodhalí tržní cenu. <i>Strategicky ekvivalentní holandské.</i> |
| <b>Vickreyova</b><br>( <i>second-price sealed bid</i> ) | jedno kolo, uzavřené obálky, nejvyšší nabídka vyhrává, platí druhou nejvyšší | <i>dominantní:</i> nabídnout <b>pravdivě</b> své ocenění                                                            | Teoreticky nejoblíbenější; Google AdWords, filatelistické aukce. <i>Ekvivalentní anglické</i> (soukromé hodnoty).             |

### Proč je pravdomlupnost dominantní strategií ve Vickreyově aukci

Ocenění  $v$ , nabídka  $b$ , nejvyšší nabídka ostatních  $p$  (neznámá). Vyhraji  $\iff b > p$ , platím  $p$ , zisk  $v - p$ .

- $b > v$  (**nadhodnocení**): oproti  $b = v$  změna jen pro  $v < p < b$  — vyhraji, platím  $p > v$ , zisk  $v - p < 0$ . *Škodí mi.*
- $b < v$  (**podhodnocení**): změna jen pro  $b < p < v$  — prohraji, ač bych vyhrál se ziskem  $v - p > 0$ . *Přijdu o zisk.*
- Nabídkou  $b \neq v$  nelze získat víc než s  $b = v$ .

$b = v$  je (slabě) dominantní bez ohledu na ostatní. Aukce **odolná vůči manipulaci** (*strategy-proof, incentive compatible*) a **efektivní** (vyhrává nejvyšší ocenění).

**Příklad — srovnání výnosů.** Ocenění  $A = 80$  Kč,  $B = 60$  Kč,  $C = 30$  Kč; ideální scénář (bez dodatečné informace, bez podvádění):

| Aukce      | Vítěz | Cena                                                              |
|------------|-------|-------------------------------------------------------------------|
| Anglická   | $A$   | $\approx 61$ (přilhuje se, dokud neodstoupí $B$ na 60)            |
| Holandská  | $A$   | 80 nebo o něco méně (79) — $A$ nesmí riskovat, že přijme až pozdě |
| FPSB       | $A$   | 80 (resp. $60 + \varepsilon$ , zná-li $A$ ocenění ostatních)      |
| Vickreyova | $A$   | 60 (druhá nejvyšší nabídka)                                       |

Vždy získá předmět nejvyšší ocenění (aukce *efektivní*); liší se cena a odhalená informace.

*Proč u FPSB 80, když se má nabízet pod oceněním?* Scénář předpokládá *nulovou* informaci o ostatních; míra podhodnocení závisí na přesvědčení o ostatních  $\Rightarrow$  FPSB (a holandská) **nemá dominantní strategii**. Zná-li  $A$  ocenění ostatních, nabídne  $60 + \varepsilon$  a výnos klesne na úroveň Vickreyovy.

### Věta o ekvivalenci výnosů (*revenue equivalence theorem*)

Vickrey (1961), Myerson, Riley–Samuelson. Předpoklady: rizikově neutrální kupující, nezávislé soukromé hodnoty ze stejného spojitého rozdělení, předmět získá nejvyšší ocenění, kupující s nejnižším možným oceněním má nulový očekávaný zisk  $\Rightarrow$  **všechny čtyři základní aukce dávají prodávajícímu stejný očekávaný výnos.**

Rozdíly v praxi = porušení předpokladů: averze k riziku (lepší FPSB/holandská), společné hodnoty a prokletí vítěze (lepší anglická — odhaluje informaci), koluze kupujících (anglická zranitelnější), náklady na komunikaci (sealed bid levnější).

## 5.13 Kombinatorické aukce

### Kombinatorická aukce

*Více předmětů současně*, nabídky na *podmnožiny* (*bundles*). Motivace: hodnoty nejsou aditivní — *komplementarita* (levá a pravá bota, sousední frekvenční bloky:  $v(\{x, y\}) > v(x) + v(y)$ ), *substitute* (dvě letenky na tentýž let:  $v(\{x, y\}) < v(x) + v(y)$ ).

- **Problém určení vítězů** (*winner determination problem*, WDP): vybrat disjunktní nabídky s maximálním součtem cen. Zobecnění množinového pakování — **NP-těžký**, i těžko aproximovatelný. Řešení: ILP solvery, větvení a meze (CABOB, CPLEX).
- **Reprezentační problém**:  $2^m - 1$  podmnožin  $\Rightarrow$  kompaktní jazyky nabídek (OR, XOR, OR-of-XOR).

## 5.14 Další typy aukcí a alokace zdrojů

- *All-pay auction* — platí všichni, vyhrává nejvyšší nabídka; model lobbingu a patentových závodů.
- *Tichá* — písemná varianta anglické; *amsterodamská* — anglická, která se při dvou zbylých zájemcích přepne na holandskou.
- *Dvojitá aukce* — nabídky kupujících i prodávajících se párují; model burzy.



## 5.15 Návrh mechanismů, VCG a vyjednávání

♦ nad rámec slidů: ve slidech není; okruh jmenuje „alokaci zdrojů“, k níž toto patří — kombinatorické aukce slidy zmiňují jednou větou]

### Návrh mechanismů (*mechanism design*)

Teorie her: *jak se agenti zachovají při daných pravidlech*. Návrh mechanismů = „obrácená teorie her“: *jaká pravidla, aby racionální sobečtí agenti sami dělali, co chceme* — typicky mluvili pravdu.

- Mechanismus = pravidlo výběru výsledku  $x(\hat{v})$  z ohlášených preferencí + platební pravidlo  $p_i(\hat{v})$ ; užitek  $u_i = v_i(x(\hat{v})) - p_i(\hat{v})$ .
- Žádoucí vlastnosti: **kompatibilita s motivací** (vyplatí se ohlásit pravdu; nejsilnější varianta *strategy-proof* — pravdomluvnost dominantní), **individuální racionalita** ( $u_i \geq 0$ ), **efektivita** (maximum sw), **vyrovnaný rozpočet**, **výpočetní zvládnutelnost**. Vše najednou nelze.
- **Princip odhalení**: ke každému mechanismu existuje *přímý* mechanismus (agenti ohlásí pravdu, výsledek stejný)  $\Rightarrow$  teorie se točí kolem Vickreyho a VCG. Bez plateb naráží na Gibbardovu–Satterthwaiteovu větu.

### VCG — Vickrey–Clarke–Groves

Zobecnění Vickreyovy aukce na kombinatorický případ: alokace  $x^*$  maximalizující  $\sum_i \hat{v}_i(x)$ ; agent  $i$  platí **externalitu způsobenou ostatním**:

$$p_i = \max_x \sum_{j \neq i} \hat{v}_j(x) - \sum_{j \neq i} \hat{v}_j(x^*).$$

Pravdomluvný, efektivní, individuálně racionální; pro jeden předmět = Vickreyova aukce. Nevýhody: WDP  $n+1$ -krát, malý (i nulový) výnos prodávajícího, zranitelnost vůči koluzi.

*Příklad*: předměty  $\{x, y\}$ ; agent 1:  $\{x\} \rightarrow 5$ , agent 2:  $\{y\} \rightarrow 4$ , agent 3:  $\{x, y\} \rightarrow 8$ . Optimální alokace  $5 + 4 = 9 > 8$ .  $p_1 = 8 - 4 = 4$  (bez agenta 1 by ostatní měli 8, se zvolenou alokací 4), zisk  $5 - 4 = 1$ ; symetricky  $p_2 = 8 - 5 = 3$ . Výnos prodávajícího  $7 < 8$ .

### Vyjednávání: monotónní protokol ústupků a Zeuthenova strategie

Aukce vyžaduje centrálního aukcionáře; alternativa = přímé **vyjednávání**. *Vyjednávací množina*: individuálně racionální a Pareto-optimální dohody.

- **Monotónní protokol ústupků**: v každém kole oba současně navrhnou dohodu. Konec, je-li návrh jednoho pro druhého alespoň tak dobrý jako jeho vlastní; jinak musí alespoň jeden *ustoupit*, jinak vyjednávání ztroskotá.
- **Zeuthenova strategie** — *kdo* ustoupí: ten s menší ochotou riskovat konflikt:

$$\text{Risk}_i = \frac{u_i(\text{vlastní návrh}) - u_i(\text{návrh protějšku})}{u_i(\text{vlastní návrh})}.$$

Ustupuje agent s *nižším* Risk, právě tak málo, aby se poměry obrátily. Dvojice Zeuthenových strategií je NE; dohoda maximalizuje součin utilit (Nashovo vyjednávací řešení).

## 5.16 Shrnutí ke zkoušce

- Koherence (výkon celku) vs. koordinace (minimalizace synchronizační režie).
- CDPS: benevolence + sdílený cíl; odlišit od PPS (homogenní procesory, paralelní algoritmus) a od MAS (vlastní cíle, konflikty, vyjednávání).
- Tři fáze CDPS: dekompozice — řešení podproblémů — syntéza. Sdílení úloh (dohoda) vs. sdílení výsledků (proaktivní/reaktivní), čtyři přínosy: jistota, úplnost, přesnost, včasnost.



- **CNET**: recognition — announcement — bidding — awarding/expediting; manažer a dodavatel, hierarchické subkontrakty; standardizován jako FIPA-Contract-Net.
- **BBS**: tabule (hierarchie abstrakce, mutex), experti (skills), arbitr (výběr experta); volná vazba mezi experty; úzké hrdlo přístupu k tabuli.
- DisCSP / DCOP: proměnné a omezení rozdělené mezi agenty, nikdo nevidí celý problém; ABT (ok? / nogood); DCOP maximalizuje sociální blahobyt.
- Sobecký agent maximalizuje *svoji* očekávanou utilitu za předpokladu, že ostatní dělají totéž; společná utilita se tím nemaximalizuje.
- Hra v normální formě, ryzí vs. smíšené strategie, dominance a dominantní strategie, iterované odstraňování dominovaných strategií.
- **Nashovo ekvilibrium** = vzájemně nejlepší odpovědi, nikdo nemá důvod se jednostranně odchýlit; naivní hledání  $m^n$ ; **Nashova věta** (existence ve smíšených strategiích); hledání PPAD-úplné.
- **Paretova optimalita** (nelze zlepšit jednomu bez přitížení jinému) vs. **sociální blahobyt** (součet utilit); maximum sw je Pareto-optimální, obráceně ne.
- **Věžňovo dilema**:  $T > R > P > S$ ;  $D$  dominantní;  $(D, D)$  jediné NE a jediné ne-Pareto-optimální;  $(C, C)$  maximalizuje sw. Tragedy of the commons. Iterované PD, stín budoucnosti, Axelrod a TFT (slušná, odvetná, odpouštějící, nezávistivá, srozumitelná).
- Umět spočítat smíšené NE pro  $2 \times 2$ : pravděpodobnosti volím tak, aby byl *protihráč* indiferentní.
- Sociální volba: plurality / IRV / sekvenční / Borda / Slater; Condorcetův paradox (cyklická kolektivní preference); kritéria Pareto, Condorcetův vítěz, IIA, univerzalita, nediktátorství; **Arrowova věta** (Pareto + IIA  $\Rightarrow$  diktatura); Gibbard-Satterthwaite (každý systém lze manipulovat).
- Aukce = mechanismus alokace vzácných zdrojů; efektivní aukce dá zdroj tomu, kdo ho oceňuje nejvíc. Dimenze: private/common value, open cry/sealed bid, first/second price, počet kol.
- Čtyři základní typy a strategie: anglická (přihazuj po  $\varepsilon$  do svého ocenění) a Vickrey (nabídní pravdivě) mají *dominantní* strategii; holandská (čkej na ocenění  $-\varepsilon$ ) a FPSB (nabídní pod oceněním) *dominantní* strategii **nemají**. Umět **důkaz pravdomluvnosti** Vickreyho.
- Ekvivalence: anglická  $\leftrightarrow$  Vickreyova, holandská  $\leftrightarrow$  FPSB. **Věta o ekvivalenci výnosů** a kdy neplatí (averze k riziku, common values, prokletí vítěze, koluze). Umět příklad s oceněními 80/60/30.
- Návrh mechanismů (obrácená teorie her), princip odhalení; kombinatorické aukce a WDP (NP-těžký); **VCG** = alokace maximalizující sw + platba rovná externalitě.
- Vyjednávání: monotónní protokol ústupků + Zeuthenova strategie (ustupuje ten s nižším rizikem).

## 6 Metody pro učení agentů a zpětnovazební učení

[♦ nad rámec slidů: celá kapitola] Přednáška učení agentů výslovně *nepokrývá*, okruh ho jmenuje. Rozsah: přehled metod, MDP a Q-learning podrobně, zbytek rámcově.

### 6.1 Proč a jak se agenti učí

Agent v MAS má být *adaptivní*; učení = změna chování na základě zkušenosti.

#### Metody pro učení agentů

- **Podle učebního signálu**: *s učitelem* (správné odpovědi — model protivníka z pozorovaných tahů, klasifikace vjemů); *bez učitele* (struktura — shlukování situací); **zpětnovazební** (jen skalární odměna z prostředí; pro autonomní agenty nejpřirozenější).
- **Imitační učení** (*learning from demonstration*): politika z trajektorií experta — *behavioral cloning*, nebo *inverzní RL* (rekonstrukce odměnové funkce z chování experta).
- **Evoluční metody**: populace kontrolérů šlechtěná podle utility — typické ladění reaktivních architektur (kap. 2); *learning classifier systems*, neuroevoluce.
- **Podle toho, co se učí**: model prostředí, model protivníka (*opponent modelling*), hodnotová funkce, přímo politika, u PRS ohodnocení plánů.
- **Individuální vs. sociální**: jen vlastní zkušenost, nebo i napodobování a sdílení zkušeností

s ostatními.

**Zpětnovazební učení** (*reinforcement learning*, RL) — nejběžnější, teoreticky nejpropracovanější; pokus–omyl podle *odměn* z prostředí.

- Kap. 1: politika  $\pi$  = mechanismus výběru akce, odměna = operacionalizace utility.
- **Jeden agent** — snazší, klasické RL (Q-learning).
- **Multiagentní RL** (*MARL*) — učení *současně*; formalizace Markovskými hrami (Nashova a Paretova optimalita); state of the art hluboké MARL (MADRL).

## 6.2 Markovský rozhodovací proces

### MDP — *Markov Decision Process*

Pětice  $\langle S, A, T, R, \gamma \rangle$ :

- $S$  — stavy;  $A$  — akce;
- $T : S \times A \times S \rightarrow [0, 1]$  — přechodová funkce,  $T(s, a, s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$ ;
- $R : S \times A \times S \rightarrow \mathbb{R}$  — okamžitá odměna za přechod;
- $0 \leq \gamma < 1$  — diskontní faktor (okamžitá vs. budoucí odměna).

**Markovská vlastnost**: další stav a odměna závisí jen na aktuálním stavu a akci, ne na historii. (Přechodová funkce  $\tau : R^A \rightarrow 2^E$  abstraktní architektury historii *má*; MDP = její markovské zjednodušení + pravděpodobnosti + odměny.)

**Politika** (*policy*)  $\pi : S \rightarrow A$  (stochasticky  $\pi(a|s)$ ). Cíl: optimální  $\pi^*$  maximalizující očekávaný diskontovaný součet odměn

$$\mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right].$$

**Hodnotová funkce**  $V^\pi : S \rightarrow \mathbb{R}$  — očekávaná utilita stavu při politice  $\pi$ ; **akční hodnotová funkce**  $Q^\pi(s, a)$  — totéž pro dvojici stav–akce.

### Bellmanovy rovnice

Pro danou politiku (*Bellmanova rovnice pro evaluaci*):

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')].$$

Pro optimální politiku (*Bellmanova rovnice optimality*):

$$V^*(s) = \max_{a \in A} \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')],$$
$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')].$$

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

**Iterace hodnot** (*value iteration*) — při známém úplném popisu MDP:

- 1: inicializuj  $V_0(s) \leftarrow 0$  pro všechna  $s$
  - 2: **repeat**
  - 3:   **for all**  $s \in S$  **do**
  - 4:      $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$
  - 5:   **end for**
  - 6: **until**  $\max_s |V_{k+1}(s) - V_k(s)| < \varepsilon$
  - 7: **return**  $\pi(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$
- Bellmanův operátor = kontrakce s koeficientem  $\gamma \Rightarrow$  konvergence ke  $V^*$  geometricky rychle.

- **Iterace politiky** (*policy iteration*): střídavě *vyhodnot* politiku (soustava lineárních rovnic) a *zlepší* ji (hladově podle  $V^\pi$ ); konečně mnoho politik  $\Rightarrow$  konvergence v konečně mnoha krocích.

### 6.3 Q-learning a SARSA

Obvykle **neznáme**  $T$  ani  $R$  — agent se učí ze zkušenosti v diskrétních krocích: *model-free* RL.

#### Q-learning (Watkins, 1989)

Tabulka  $Q(s, a)$  — odhad diskontovaného součtu budoucích odměn po akci  $a$  ve stavu  $s$ . Po přechodu  $s \xrightarrow{a} s'$  s odměnou  $r$ :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ \underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{cíl (TD target)}} - Q(s, a) \right], \quad 0 \leq \alpha \leq 1 \text{ je rychlost učení,}$$

ekvivalentně  $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$ .

**Off-policy:** cíl s  $\max_{a'}$  — učí se hodnotu *optimální* politiky nezávisle na politice, kterou agent jedná. Konvergence  $Q \rightarrow Q^*$ : každá dvojice  $(s, a)$  navštívena nekonečněkrát,  $\alpha$  vhodně klesá.

#### Dilema průzkumu a využití; $\varepsilon$ -greedy

*Exploit* (využívat naučené) vs. *explore* (zkoušet nové akce). Standardní řešení:

$$\pi(s) = \begin{cases} \text{náhodná akce z } A & \text{s pravděpodobností } \varepsilon, \\ \arg \max_a Q(s, a) & \text{s pravděpodobností } 1 - \varepsilon. \end{cases}$$

$\varepsilon$  v čase klesá (*annealing*). Alternativy: softmax/Boltzmann  $\pi(a|s) \propto e^{Q(s,a)/\tau}$ , optimistická inicializace  $Q$ , UCB (horní meze spolehlivosti).

**Konkrétní krok Q-learningu.**  $\alpha = 0,5$ ,  $\gamma = 0,9$ ; ve stavu  $s_1$  akce  $a_1$ , odměna  $r = 10$ , přechod do  $s_2$ . Hodnoty:  $Q(s_1, a_1) = 4$ ,  $Q(s_2, a_1) = 6$ ,  $Q(s_2, a_2) = 8$ .

$$\max_{a'} Q(s_2, a') = \max(6, 8) = 8,$$

$$\text{TD cíl} = r + \gamma \max_{a'} Q(s_2, a') = 10 + 0,9 \cdot 8 = 17,2,$$

$$\text{TD chyba} = 17,2 - Q(s_1, a_1) = 17,2 - 4 = 13,2,$$

$$Q(s_1, a_1) \leftarrow 4 + 0,5 \cdot 13,2 = \mathbf{10,6}.$$

#### SARSA

*State–Action–Reward–State–Action*: v aktualizaci *skutečně zvolená* další akce  $a'$  (touž politikou, kterou agent jedná):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)].$$

**On-policy:** učí se hodnotu prováděné politiky včetně náhodného průzkumu  $\Rightarrow$  *opatrnější*. „Chůze po útesu“ (*cliff walking*): Q-learning — optimální cesta těsně u propasti, při  $\varepsilon$ -greedy občas spadne; SARSA — bezpečnější cesta dál od okraje, započítá riziko vlastního průzkumu. Zobecnění obou: TD( $\lambda$ ) s *eligibility traces* (kredit rozprostřen na více předchozích kroků).

## 6.4 Metody založené na politice

### Policy gradient a actor–critic

Optimalizace *přímo politiky*: parametrizace  $\pi(a | s, z)$ , hledáme  $z^*$  gradientním výstupem po očekávaném výnosu. **REINFORCE** (Williams, 1992) — odhad gradientu z Monte Carlo simulací celých epizod:

$$z_{t+1} = z_t + \alpha G_t \nabla_z \ln \pi(A_t | S_t, z_t),$$

$G_t$  = výnos od času  $t$ ; intuice: zvyš pravděpodobnost akcí následovaných vysokým výnosem.

- Výhody proti hodnotovým metodám: *spojité* akční prostory; přirozeně *stochastické* politiky (nutné ve hrách se smíšenými ekvilibrii).
- Nevýhoda: vysoký rozptyl odhadu gradientu.

**Actor–critic** — kombinace: *actor* = politika, *critic* = hodnotová funkce (snižuje rozptyl).

**Funkce výhody**  $A(s, a) = Q(s, a) - V(s)$ : gradient škálován relativní výhodností akce místo výnosu. Varianty: A3C, A2C, PPO, SAC, DDPG.

**Hluboké RL: DQN** — tabulka  $Q$  nahrazena neuronovou sítí; triky *experience replay* (rozbití korelací mezi vzorky), *target network* (stabilizace cíle).

## 6.5 Multiagentní zpětnovazební učení

### Markovská hra (*Markov game*, stochastická hra)

MDP pro  $N$  agentů:  $\langle N, S, (A_i)_{i \in N}, T, (R_i)_{i \in N}, \gamma \rangle$ . **Sdružený akční prostor**  $\mathbf{A} = A_1 \times \dots \times A_N$ ; přechod závisí na akcích *všech*; každý agent má *vlastní*  $R_i$  a  $\pi_i$ , jeho hodnotová funkce závisí na politikách všech.

**Vazba na teorii her**: politika agenta = *nejlepší odpověď* na sdružené politiky ostatních; sdružené politiky mohou být v **Nashově ekvilibriu** a **Pareto-optimální**. Speciální případy:  $N = 1$  — MDP; jeden stav — hra v normální formě;  $N = 2$  s nulovým součtem — *minimax* hra.

**Minimax-Q** (Littman, 1994) — dva agenti, nulový součet; místo  $\max_{a'} Q(s', a')$  hodnota smíšeného minimaxového ekvilibria matice  $Q(s', \cdot, \cdot)$  (lineární program). **Nash-Q** — totéž pro obecný součet, konverguje jen za silných předpokladů.

### Proč je MARL těžké

- Politiky aktualizovány **současně**  $\Rightarrow$  prostředí z pohledu agenta **nestacionární** („pohyblivý cíl“); porušen základní předpoklad konvergence RL.
- Sdružený akční prostor roste **exponenciálně** s počtem agentů.
- Jen **částečná pozorovatelnost** (Dec-POMDP; optimální řešení NEXP-úplné).
- **Přiřazení zásluh**: při společné odměně nejasné, kdo přispěl.

**Protipatření**: *centralizované učení s decentralizovaným prováděním* (CTDE) — kritik při tréninku vidí stav i akce všech, herec za běhu jen svá lokální pozorování (MADDPG, COMA, QMIX); sdílení parametrů mezi homogenními agenty; *opponent modelling*.

**Self-play**. Učení hrou proti sobě (či dřívějším verzím sebe); v hrách s nulovým součtem spolehlivě silné strategie (TD-Gammon, AlphaZero). Riziko: *cyklické* preference strategií  $\Rightarrow$  naivní self-play osciluje; proto ligy protivníků a populační metody.

## 6.6 Shrnutí ke zkoušce

- Metody učení agentů: s učitelem / bez učitele / zpětnovazební; imitační učení a inverzní RL; evoluční metody (LCS, neuroevoluce); co se agent učí (model prostředí, model protivníka, politiku); individuální vs. sociální učení. RL je nejběžnější.
- **MDP** =  $\langle S, A, T, R, \gamma \rangle$ , Markovská vlastnost, politika  $\pi$ , hodnotové funkce  $V^\pi, Q^\pi$ ; Bellmanova rovnice optimality; value iteration (kontrakce, konverguje) a policy iteration.

- **Q-learning:**  $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ , off-policy, tabulkový; umět spočítat jeden krok s čísly.
- **SARSA:**  $\dots + \alpha[r + \gamma Q(s', a') - Q(s, a)]$ , on-policy, opatrnější (cliff walking).  $\epsilon$ -greedy jako řešení dilematu průzkum/využití.
- **Policy gradient** (REINFORCE,  $\nabla \ln \pi$  krát výnos) a **actor-critic** (herec = politika, kritik = hodnotová funkce, výhoda  $A = Q - V$ ); DQN (replay buffer, target network).
- **Markovská hra** = MDP pro  $N$  agentů se sdruženým akčním prostorem a individuálními odměnami; nejlepší odpověď, Nashovo ekvilibrium, Pareto-optimalita. **Minimax-Q** (nulový součet, LP v každé aktualizaci), Nash-Q (obecný součet).
- Problémy MARL: nestacionarita, exponenciální sdružený akční prostor, částečná pozorovatelnost, přiřazení zásluh; řešení CTDE. Self-play a AlphaZero.

## 7 Metodologie návrhu, jazyky a prostředí MAS

### 7.1 Agentově orientované softwarové inženýrství

Proč nestačí OO metodologie (UML):

- agenti *autonomní* — nelze přímo volat jejich metody;
- vlastní cíle;
- vztahy *organizační* (role, protokoly, závazky), nikoli strukturální.

Odpověď: metodologie **AOSE** (*agent-oriented software engineering*).

#### Gaia (Wooldridge, Jennings, Kinny, 2000)

Jedna z nejcitovanějších AOSE metodologií. Metafora: MAS = **organizace**, agenti = **role**. Postup: požadavky  $\rightarrow$  analýza  $\rightarrow$  návrh; implementace až na konkrétní platformě.

**Analýza** — dva abstraktní modely:

- *Model rolí (roles model)*; role = čtyři atributy: *permissions* — zdroje, které smí číst/měnit; *responsibilities* — co má zajistit: *liveness* („něco dobrého se stane“, regulární výraz nad aktivitami a protokoly) a *safety* („nic špatného se nestane“, invarianty); *activities* — výpočty prováděné rolí samotnou; *protocols* — interakce s jinými rolemi.
- *Model interakcí*: protokoly mezi rolemi — účel, iniciátor, respondér, vstupy, výstupy.

**Návrh** — tři konkrétní modely:

- *model agentů*: role  $\rightarrow$  typy agentů, počty instancí;
- *model služeb*: funkce poskytované rolí — vstupy, výstupy, předpoklady, efekty;
- *model známostí (acquaintance model)*: orientovaný graf, kdo s kým komunikuje — kontrola úzkých hrdel a přílišné provázanosti.

**Omezení:** uzavřený systém, kooperativní agenti, statická organizace; uvolňují rozšíření *ROAD-MAP* a *Gaia v.2*.

### 7.2 Standard FIPA a architektura platformy

#### Referenční architektura agentní platformy podle FIPA

- **AMS** (*Agent Management System*) — povinná; „matrka“ platformy: evidence agentů, jednoznačné *AID* (agent identifier), životní cyklus podle stavového diagramu FIPA (*initiated*  $\rightarrow$  *active*  $\leftrightarrow$  *waiting/suspended*, u mobilních agentů navíc *transit*; operace create, suspend, resume, terminate), bílé stránky (*white pages*).
- **DF** (*Directory Facilitator*) — žluté stránky (*yellow pages*): registrace nabízených služeb, vyhledávání jejich poskytovatelů.
- **MTS** (*Message Transport Service*) — doručování ACL zpráv v rámci platformy i mezi platformami (protokoly IIOP, HTTP; kódování bitefficient, XML, string).
- **ACC** (*Agent Communication Channel*) — rozhraní k MTS.

Standard dále definuje: jazyk ACL, obsahové jazyky (SL), interakční protokoly, správu ontologií.

### 7.3 Jazyky a prostředí

♦ nad rámec slidů: AGENT-0 a Concurrent MetateM ve slidech nejsou; JADE a Jason ano]

- **Agentově orientované programování** (Shoham 1993) — historické východisko: stav programu = *mentální kategorie* (přesvědčení, závazky, schopnosti), program = množina *pravidel závazků*.
- **AGENT-0** — první jazyk AOP.
- **Concurrent MetateM** — jiná cesta: agent = *přímo spustitelná specifikace* v temporální logice.
- Oba navazují na deduktivní agenty (odst. 2.1); prakticky nejúspěšnější agentní jazyk dnes: Agent-Speak/Jason.

| Systém  | Typ                             | Charakteristika                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JADE    | Java, FIPA platforma            | <i>Java Agent DEvelopment Framework</i> ; jedna z nejrozšířenějších platforem, plně FIPA. Agent = třída dědící z <b>Agent</b> , metoda <b>setup()</b> ; chování = objekty <b>Behaviour</b> ( <b>OneShot</b> , <b>Cyclic</b> , <b>Ticker</b> , <b>FSMBehaviour</b> , <b>SequentialBehaviour</b> ), kooperativní (nepreemptivní) plánovač v jednom vlákně agenta. Zprávy: <b>ACLMessage</b> , <b>send()</b> / <b>blockingReceive()</b> , <b>MessageTemplate</b> . Hotové protokoly ( <b>ContractNetInitiator</b> , <b>AchieveREResponder</b> ), ladící nástroje (RMA, Sniffer, Introspector), kontejnery, mobilita agentů.       |
| Jason   | interpret Agent-Speak(L) v Javě | Referenční implementace <b>AgentSpeak</b> = <b>BDI</b> jazyk (logické programování + BDI), přímý potomek PRS. Program = počáteční <i>beliefs</i> , <i>rules</i> , <i>knihovna plánů</i> tvaru <b>spouštěč : kontext</b> <- <b>tělo</b> . Spouštěč = <i>událost</i> : <b>+b</b> přidání přesvědčení, <b>!g</b> přidání cíle, <b>-!g</b> selhání cíle. Cyklus interpretu: vnímej → aktualizuj <i>beliefs</i> → vyber událost → <i>relevantní</i> plány (podle spouštěče) → <i>aplikovatelné</i> (podle kontextu) → vytvoř záměr → vykonej krok. Jason + <b>CARtAgO</b> (prostředí) + <b>Moise</b> (organizace) = <i>JaCaMo</i> . |
| NetLogo | prostředí pro ABM               | Jazyk odvozený od Logo pro <i>agent-based modeling</i> . Entity: <i>turtles</i> (mobilní agenti), <i>patches</i> (buňky prostředí), <i>links</i> , <i>observer</i> . Knihovna modelů (Segregation, Wolf Sheep Predation, Ants, Flocking), <i>BehaviorSpace</i> pro parametrické studie. Emergentní chování a výuka, ne FIPA komunikace.                                                                                                                                                                                                                                                                                        |
| SPADE   | Python, XMPP                    | <i>Smart Python Agent Development Environment</i> . Komunikace standardním <b>XMPP</b> místo vlastního MTS; agent = třída s <i>behaviours</i> ( <b>CyclicBehaviour</b> , <b>PeriodicBehaviour</b> , <b>FSMBehaviour</b> ); asyncio, webové rozhraní; populární díky napojení na Python ekosystém ML.                                                                                                                                                                                                                                                                                                                           |

#### Jason podrobněji (látka cvičení NAIL106)

##### Jazyk.

- Spouštěče **+b**, **-b**, **!g**, **-!g**; navíc *testovací cíl* **?g** — dotaz do báze *beliefs*, unifikací naplní proměnné.
- Tělo plánu mění *beliefs* přímo: **+b** přidání, **-b** odebrání, **-+b** aktualizace.
- **Dvě negace**: **not b** = negace jako neúspěch („agent nemá důvod věřit **b**“); **~b** = silná negace („agent věří, že **b** neplatí“).
- *Anotace* *beliefs*, hlavně zdroj: **likes(a,b)[source(martin)]** — zdroj = *vjem* (**percept**), agent sám (**self**), nebo jiný agent.

**Komunikace.** Interní akce **.send(příjemce, performativ, obsah)**; performativy **tell**, **untell**, **achieve**, **askOne**, ... = řečové akty ve smyslu KQML/FIPA-ACL (kap. 4). **tell** přidá příjemci



belief s anotací `source(odesílatel)`, `achieve` mu přidá cíl. Ve slidech pivní robot z odd. 2: `.send(supermarket, achieve, order(beer,5))`.

**Prostředí a projekt.** Prostředí = třída v Javě dědící z `Environment`: přepsat `executeAction(agent, akce)`, vjemy agentům přes `addPercept`; pro mřížkové světy hotový `GridWorldModel`. Projektový soubor `.mas2j`: seznam agentů (a počty instancí), třída prostředí, infrastruktura.

**Přizpůsobení interpretu.**

- Tři volby cyklu (vyber událost / aplikovatelný plán / záměr) = tři *selekční funkce* `selectEvent`, `selectOption`, `selectIntention`; přepis děděním z `Agent` (např. prioritizace typu události).
- Dědit lze celou architekturu `AgArch` (`perceive`, `checkMail` — vlastní model světa z vjemů a zpráv).
- Vlastní `BeliefBase` (např. automatická náhrada duplicitních beliefs).
- Výpočetně těžké podprogramy (typicky hledání cest) = vlastní interní akce děděním z `DefaultInternalAction`.

**BattleCode** (slidy seminářů kurzu vedle JADE a AgentSpeak/Jasonu): programátorská soutěž MIT — bitevní strategie + softwarové inženýrství + UI; tým píše program autonomně řídící každého robota své armády zvlášť (decentralizované řízení, omezený výpočet na tah, omezená komunikace) = MAS v malém jako cvičiště.

**Další prostředí:**

- *Jadex*, *JACK*, *JAM* — další implementace BDI/PRS (Jadex = BDI vrstva nad JADE).
- *MASON*, *Repast*, *GAMA* — simulační platformy pro agentní modely.
- Multiagentní zpětnovazební učení (kap. 6): **Gymnasium** (nástupce OpenAI Gym) — standardní API jednoagentního RL: `reset()`, `step(akce)` → pozorování, odměna, příznak konce; **Petting-Zoo** — multiagentní obdoba, dvě API: *AEC* (agenti táhnou střídavě) a *Parallel* (všichni současně), hotová prostředí (MPE, multiplayer Atari, deskové hry); **OpenSpiel** (DeepMind) — kolekce her v extenzivní i normální formě (nulový i obecný součet, imperfektní informace) a referenčních algoritmů (CFR, fictitious play, MCTS) pro učení ve hrách.

## 7.4 Shrnutí ke zkoušce

- Proč nestačí OO metodologie: autonomie, vlastní cíle, organizační (nikoli strukturální) vztahy.
- **Gaia**: organizace + role; analýza = model rolí (permissions, responsibilities s liveness a safety, activities, protocols) + model interakcí; návrh = model agentů, služeb a znalostí. Předpoklad kooperativních agentů a statické organizace.
- **FIPA platforma**: AMS (bílé stránky, AID, životní cyklus: initiated, active, waiting, suspended, transit), DF (žluté stránky, služby), MTS/ACC (doručování ACL zpráv).
- **JADE** = Java, FIPA, Agent + Behaviour + ACLMessage, hotové protokoly a ladicí nástroje. **Jason/AgentSpeak** = BDI jazyk, plán spouštěč : `kontext <- tělo`, potomek PRS. **NetLogo** = ABM simulace (turtles/patches/links). **SPADE** = Python + XMPP.
- **Jason prakticky**: testovací cíl `?g`, aktualizace `-+b`; `not` (negace jako neúspěch) vs. `~` (silná negace); anotace `[source(...)]`; `.send` s performativy `tell/achieve` = řečové akty; prostředí v Javě (`executeAction`, `addPercept`), projekt `.mas2j`; přizpůsobení přes selekční funkce cyklu, `AgArch` a vlastní interní akce.

## Závěrem: jak na zkoušku

Zkoušející ověřuje *souvislosti*. Čtyři osy otázek:

1. **Reaktivita vs. proaktivita** — definice inteligentního agenta → commitment strategie (bold vs. cautious), parametry  $\gamma/\phi$  v síti Maes → vrstvy hybridních architektur.

2. **Individuální vs. kolektivní racionalita** — sobecký agent  $\rightarrow (D, D)$  ve vězňově dilematu, tragédie obecní pastviny, taktické hlasování  $\rightarrow$  potřeba mechanism designu (Vickrey, VCG): návrh *pravidel*, za nichž se sobeckému agentovi vyplatí pravdomluvnost.
3. **Symbolická vs. subsymbolická reprezentace** — FOL, STRIPS, ontologie, ACL vs. subsumpce, aktivacíní sítě, neuronové sítě v RL; praxe hybridní.
4. **Teorie vs. praxe** — elegantní, ale nekonstruktivní definice (optimální agent, Nashova věta, Arrowova věta) vs. použitelné algoritmy ( $A^*$ , CBS, Q-learning, CNET, JADE). Složitost: PPAD-úplnost hledání NE, NP-těžkost WDP i optimálního MAPF.

Časté drobnosti: *desire* vs. *intention*; proč efektem **inform** *není*, že příjemce uvěří; proč je  $(D, D)$  jediné *ne*-Pareto-optimální řešení vězňova dilematu; proč je pravdomluvnost dominantní ve Vickreyově aukci; off-policy Q-learning vs. on-policy SARSA; co dělá CBS na vysoké a co na nízké úrovni.